基于几何解析法与粒子群算法的烟幕干扰弹的投放策略

摘要

本文针对无人机投放烟幕干扰弹的策略问题，通过分析导弹、烟幕弹与无人机的空间运动关系，建立了基于解析法的三维空间运动模型与单目标协同优化模型，并采用解析法、二分法及粒子群优化算法对模型进行求解与优化。

针对问题一，在给定无人机 FY1 飞行速度、投弹及起爆时间条件下，通过平抛运动确定烟幕弹运动轨迹，导弹 M1 位置由单位方向向量和速度计算。根据速度，位移和时间关系建立导弹、烟雾弹与无人机的三维空间运动模型。将真目标分成三层，每层等弧度取关键点与导弹坐标相连，通过几何解析法联立线段和云团球体方程，求解交点个数，判断导弹与关键点位连线是否被烟幕球遮挡。采用 0.05s 粗步长扫描定位状态突变区间和二分法精确定位遮挡转换时刻，整合遮蔽区间并计算总有效遮蔽时长为 1.38s。

针对问题二，基于问题一导弹、烟雾弹与无人机的三维空间运动模型，以无人机飞行速度和方向，起爆点位置，投放时刻和起爆延迟作为决策变量，以最大有效总遮蔽时长为目标函数，建立单目标优化模型。遮挡判定仍通过解析法实现。采用粒子群优化算法计算适应度并迭代优化，最终求解出 FY1 的飞行方向为 178.8°，飞行速度 123.56m/s，烟幕干扰弹投放点$(17772.72,0.56,1800)m$，烟幕干扰弹起爆点$(17343.49,9.32,1740.84)m$，最长遮蔽时长为 4.51s。

针对问题三，无人机 FY1 投放烟幕干扰弹增至三枚，在问题二的模型基础上，增加了投放间隔这一决策变量，建立以最大有效总遮蔽时长为目标函数的单目标优化模型。仍通过几何解析法判断是否全遮挡，采用粒子群优化算法迭代优化，最终输出最优参数与遮蔽区间。其中无人机的飞行方向为 179.57°，三枚烟幕干扰弹投放点的 x 坐标分别为 17799.86m、17659.87m、17238.15m，最长有效遮蔽时长为 6.51s，其余数据保存至 *result1.xlsx*。

针对问题四，在问题二的基础之上，无人机增至三架，仍以最大有效遮蔽时长为目标函数，以三架无人机的速度大小和方向、烟幕弹投放时刻和延迟作为约束条件，使有效遮挡时长并集最大，对此建立了协同飞行模型和单目标优化模型。通过二次方程判别式判断是否全遮挡。采用粒子群优化算法，迭代出最长有效遮挡时长为 10.80s，其中 FY2 的速度和方向分别为 86.64m/s、251.6°，投弹坐标为$(11736.94,608.07,1400)$，其余数据保存至 *result2.xlsx*。

针对问题五，基于问题三的模型，无人机数量增至五架，以无人机初始位置到导弹飞行轨迹的最短距离为核心指标，无人机优先干扰核心指标最短的导弹。对无人机飞行速度和方向，导弹投放时刻、起爆延迟、起爆点位置和投放间隔进行约束，以总遮蔽时长最大化为目标，构建多机多弹协同模型，对每架无人机分别使用模拟退火算法，求解出所有导弹的最长有效遮蔽时长为 18.757s，FY5 的方向和速度分别为 118.24°、132.293m/s，投放的第二枚烟幕弹的投放坐标和起爆坐标分别为$(12207.807,-525.058,1300)$、$(12130.233,-380.628,1292.475)$，其余数据保存至 *result3.xlsx*。

关键词：烟幕干扰弹 二分法 三维空间运动模型 粒子群算法 模拟退火算法

一、问题重述

1.1 问题背景

烟幕干扰弹多依靠化学燃烧或爆炸来分散，形成烟幕或气溶胶云团，在目标前方特定空域实现遮蔽以干扰敌方导弹，具备成本低、效费比高的优势。根据现有技术能精准控制烟幕弹抵达的位置，并且准确控制起爆时间。

1.2 问题提出

问题一：动用无人机 FY1 投放 1 枚烟幕干扰弹来干扰导弹 M1。FY1 以 120m/s 的速度向假目标方向飞行，在接收到任务 1.5s 后就投放该干扰弹，且爆炸延时为 3.6s。要求计算这种情况下，该烟幕干扰弹对导弹 M1 的有效遮蔽时长。

问题二：为使干扰弹对 M1 的遮蔽时长达到最长，使用无人机 FY1 投放 1 枚烟幕干扰弹对导弹 M1 进行干扰。需明确 FY1 的飞行方向、飞行速度，以及烟幕干扰弹的投放位置和起爆位置，形成有效的遮蔽区间。

问题三：让无人机 FY1 投放 3 枚烟幕干扰弹对导弹 M1 进行干扰。需要制定出这 3 枚干扰弹的投放策略，并且要把最终策略结果存储到 *result1.xlsx* 文件中。

问题四：调用 FY1、FY2、FY3 这 3 架无人机，每架无人机各投放 1 枚烟幕干扰弹，共同对导弹 M1 实施干扰。使烟幕干扰弹对 M1 的遮蔽时长最长，需设计出这 3 枚烟幕干扰弹各自的投放策略，同时将策略结果保存至 *result2.xlsx* 文件。

问题五：运用 5 架无人机执行干扰任务，每架无人机最多可投放 3 枚烟幕干扰弹，目标是对 M1、M2、M3 这 3 枚来袭导弹进行干扰。需制定出所有烟幕干扰弹的投放策略，并将策略结果保存到 *result3.xlsx* 文件。

二、问题分析

对于问题一：问题一核心是在固定参数下，精准计算单枚烟幕干扰弹对导弹 M1 的有效遮蔽时长，需围绕时空轨迹建模和遮蔽判定逻辑展开，同时进行模型的误差分析。无人机 FY1 初始位置（17800, 0, 1800），以 120m/s 朝假目标飞行，1.5s 后投放烟幕弹，按匀速直线运动可算出投放点。烟幕弹脱离后水平继承无人机速度，竖直受重力自由落体，3.6s 起爆后以 3m/s 匀速下沉，结合平抛运动与匀速下沉公式，能推导起爆点及不同时刻烟幕球位置。导弹 M1 从（20000, 0, 2000）以 300m/s 直指原点，通过方向向量归一化计算实时位置，若命中原点则位置固定。真目标为底面圆心（0, 200, 0）、半径 7m、高 10m 的圆柱，选取若干个关键点位，通过解析法，将 M1 与点位连线代入烟幕球方程，用判别式判断是否全相交，全相交则判定有效遮蔽。时间统计采用 0.05s 粗步长扫描和二分法精修，定位状态突变区间，整合遮蔽区间并补全边界。

对于问题二：问题二是单枚烟幕弹遮蔽时长最大优化，需明确决策变量、约束、目标函数，其中约束为速度范围、起爆点 z 坐标合理，避免落地或者过高失效，时间非负。目标函数为烟幕弹起爆后 20s 内有效遮蔽总时长，计算逻辑同问题一，需通过关键点位线段、球体相交判定遮蔽状态，统计总时长。

对于问题三：问题三是无人机 FY1 投放 3 枚烟幕弹干扰 M1，核心是多弹时空协同，通过参数约束与算法优化最大化总遮蔽时长。从参数关联看，无人机速度、方向共享，3 枚弹投放间隔不低于 1s，起爆延迟独立非负，且起爆点 z 坐标需合理。无人机任意时刻位置由飞行参数与时间确定，第 k 枚弹投放点为对应投放时刻无人机位置；烟幕弹脱离后平抛运动，结合起爆延迟算起爆点，起爆后以 3m/s 下沉，各弹有效时间均为起爆后 20s，需单独计算烟幕球位置；导弹 M1 轨迹和遮蔽判定同问题一，合并 3 枚导弹遮蔽区间，总时长最大为目标函数。

对于问题四：问题四是 3 架异构无人机 FY1、FY2、FY3 各投 1 枚弹干扰 M1，核心是多机参数差异化与时空配合，通过协同建模与算法优化提升遮蔽效果。三架无人机约束条件相同，且无人机 FY1、FY2、FY3 的坐标分别表示为 $(17800,0,1800)$、$(12000,1400,1400)$、$(6000,-3000,700)$，需单独计算无人机飞行轨迹。在轨迹建模时，每架无人机投放点为对应投放时刻位置，烟幕弹平抛后结合起爆延迟计算起爆点，起爆后匀速下沉，各弹有效时间 20s。导弹 M1 轨迹不变，确保多机干扰目标统一，提高遮蔽导弹时长。

对于问题五：问题五是五架无人机投放最多 3 枚烟幕弹干扰 3 枚导弹，无人机初始位置到导弹飞行轨迹的最短距离为核心指标，距离越近的无人机优先分配至对应导弹。以无人机飞行速度和方向，导弹投放时刻、起爆延迟、起爆点位置和投放间隔进行约束，总遮蔽效能最大化为目标，构建多机多弹协同模型，通过参数约束与算法优化最大化总遮蔽时长。无人机投放导弹间隔不低于 1s，起爆延迟非负，且起爆点 z 坐标需合理。三枚导弹轨迹和遮蔽判定同问题三，把五架无人机分别对相应导弹的有效遮蔽时段进行合并，总有效遮蔽时长最大为目标函数。

三、模型假设

(1) 假设无人机受领任务后调整飞行方向的过程瞬时完成。
(2) 假设烟幕干扰弹被投放后水平方向完全继承无人机的速度，无阻力造成损失。
(3) 假设来袭导弹 M1、M2、M3 始终沿直线匀速飞行。
(4) 假设烟幕云团起爆后瞬时形成标准球体。
(5) 假设不考虑天气、空气湿度、风速、气压等环境因素。

四、符号说明

符号	说明	单位
P_{M1}	导弹的实时位置	\
P_y	烟幕球球心的实时位置	\
S	相对敏感性系数	\
R_y	烟幕弹云烟半径	m
$\vec{u}$	导弹到原点的单位方向向量	\
P	圆柱关键点	\
d_i	第 i 枚烟幕弹起爆延迟	s
t_i	第 i 枚烟幕弹投放时刻	s
$z_{k,i}$	第 k 架无人机投放第 i 枚导弹起爆点高度	m
$t_{r,max}$	投放时刻上限	s
$t_{d,max}$	起爆延迟上限	s

五、模型的建立与求解

5.1 问题一模型的建立和求解

已知无人机的飞行速度以及方向，核心是在特定的参数下计算烟幕弹爆炸产生的云团对真目标的有效遮挡时长。结合几何知识，从物体时空轨迹建模和完全遮挡判定逻辑进行建模与求解。

5.1.1 问题一模型的建立

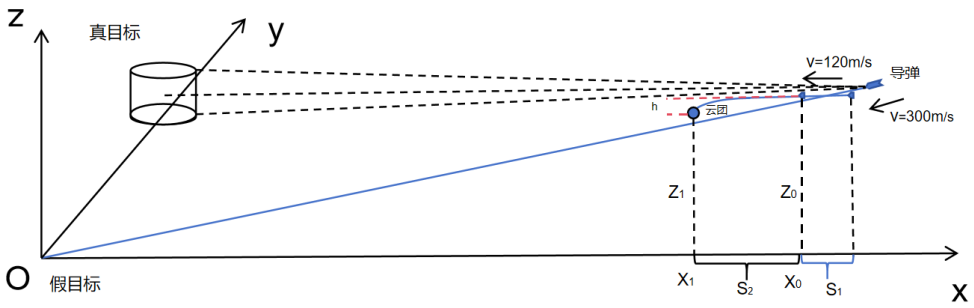


图 1 导弹、烟幕弹与无人机的三维空间运动示意图

①无人机投弹模型

受领任务后经过时间t_1投放烟幕弹，此时记无人机飞行路程s_1、飞行速度v_1，则：

$$s_1=v_1t_1 \tag{1}$$

投放烟幕干扰弹后，烟幕干扰弹在爆炸前，即时间t_2内保持原有水平速度，由于重力加速度，有竖直向下的速度，水平位移s_2和竖直位移h分别为：

$$\begin{cases} s_2 = v_1t_2 \\ h = \dfrac{1}{2}gt_2^{\,2} \end{cases} \tag{2}$$

爆炸时，记此时烟幕干扰弹爆炸形成的球体球心坐标为(x_1,y_1,z_1)，无人机初始时刻坐标为(x_0,y_0,z_0)，则有：

$$\begin{cases} x_1 = x_0 - s_1 - s_2 \\ z_1 = z_0 - h \end{cases} \tag{3}$$

由题可知$x_0=17800$，$z_0=1800$，代入上式知$x_1=17188$，$z_1=1736.496$，即爆炸时刻烟幕干扰弹形成的球体球心为$(17188,0,1736.496)$。

②导弹飞行模型

导弹 M1 以$300m/s$匀速直线飞行，方向直指假目标，即原点(0,0,0)，初始位置(20000,0,2000)。从 M1 初始位置指向原点，方向向量为：

$$\vec{d}_{M1}=(0-x_{M1,0},0-y_{M1,0},0-z_{M1,0}) \tag{4}$$

计算方向向量长度 M1 初始坐标到原点的直线距离：

$$L_{M1}=\sqrt{d_{M1,x}^2+d_{M1,y}^2+d_{M1,z}^2} \tag{5}$$

计算单位方向向量，消除长度影响：

$$\vec{u}_{M1}=\left(\frac{d_{M1,x}}{L_{M1}},\frac{d_{M1,y}}{L_{M1}},\frac{d_{M1,z}}{L_{M1}}\right) \tag{6}$$

导弹 M1 速度 $v_{M1}=300m/s$，飞行距离为：

$$s_{M1}(t)=300\cdot t \tag{7}$$

t 时刻导弹 M1 的实时位置公式：

$$P_{M1}(t)=\left(x_{M1,0}+u_{M1,x}\times s_{M1}(t),y_{M1,0}+u_{M1,y}\times s_{M1}(t),z_{M1,0}+u_{M1,z}\times s_{M1}(t)\right) \tag{8}$$

若 $s_{M1}(t)\geq L_{M1}$，判定导弹 M1 命中假目标，位置固定在(0,0,0)。

t 时刻烟幕球水平方向 x 固定为 $x_1=17188$，y 固定为 0；竖直方向上匀速下沉，初始 $z_1=1736.496$，下沉速度 $3m/s$，故 t 时刻烟幕球球心位置为：

$$P_y(t)=\left(x_1,0,z_1-3.0\times(t-5.1)\right) \tag{9}$$

其中烟幕弹有效遮挡目标对应的时间约束为 $5.1s\leq t\leq(5.1+20)s$。

③遮挡判定模型

当且仅当导弹 M1 与圆柱所有关键点位的视线，即所有的连线线段均与烟幕球相交，真目标圆柱被完全遮挡。定义导弹和烟幕球球心的实时位置为：

$$\begin{cases}P_{M1}(t)=(L_x,L_y,L_z)\\P_y(t)=(S_x,S_y,S_z)\end{cases} \tag{10}$$

烟幕弹云团遮挡分为两种情况：

若导弹在烟幕球内，即导弹与球心距离小于等于球半径时，所有视线均被挡，公式为：

$$\sqrt{(L_x-S_x)^2+(L_y-S_y)^2+(L_z-S_z)^2}\leq 10 \tag{11}$$

示意图如下：

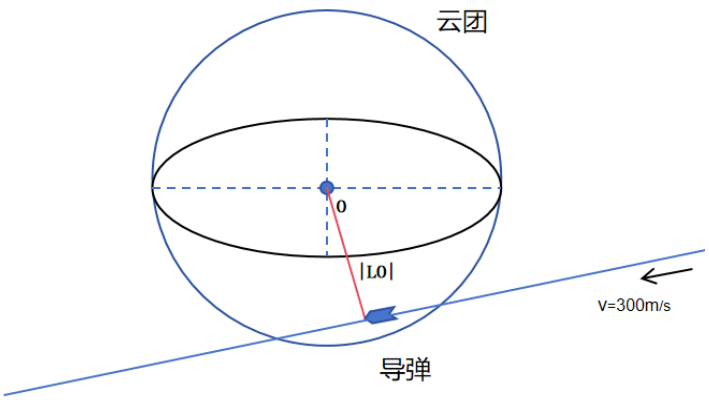


图 2 导弹在烟幕球内遮挡示意图

若导弹在烟幕球外，为确保关键射线全被挡则圆柱全被挡，选择遮挡圆柱边缘、轴线、中间区域的 27 个关键点位 P，如顶部与底部的左、右边缘等。

关键射线的起点 $L=P_{M1}(t)$ ，云团球心 $O=P_y\left(t\right)$，终点为关键点位 P，判定线段 LP 是否与烟幕弹云团相交，线段向量可表示为：

$$\overrightarrow{LP}=(P_x-L_x,P_y-L_y,P_z-L_z) \tag{12}$$

球心到线段起点向量：

$$\overrightarrow{OL}=(L_x-S_x,L_y-S_y,L_z-S_z) \tag{13}$$

线段 LP 参数方程为：

$$Q(k)=L+k\cdot\overrightarrow{LP} \tag{14}$$

其中 $k\in[0,1]$，且 $k=0$ 和 $k=1$ 时分别对应点 L 和点 P 。参数方程 $Q\left(k\right)$ 的坐标 $\left(x,\ y,\ z\right)$ 为：

$$\begin{cases}x=L_x+k\left(P_x\ -\ L_x\right)\\ y=L_y+k\left(P_y-L_y\right)\\ z=L_z+k\left(P_z-\ L_z\right)\end{cases} \tag{15}$$

已知云团圆心坐标，可写出球体方程：

$$(x-S_x)^2+(y-S_y)^2+(z-S_z)^2=R^2 \tag{16}$$

将公式（15）代入到公式（16）中，展开并整理得关于 k 的一元二次方程：

$$Ak^2+Bk+C=0 \tag{17}$$

用判别式 Δ 是否大于 0 来判别线段 LP 与球体是否相交，若所有关键射线均满足线段 LP 与烟幕球相交，则圆柱完全被遮挡；若存在任意一条关键射线不相交，则圆柱未完全被遮挡。圆柱完全被遮挡示意图如下：

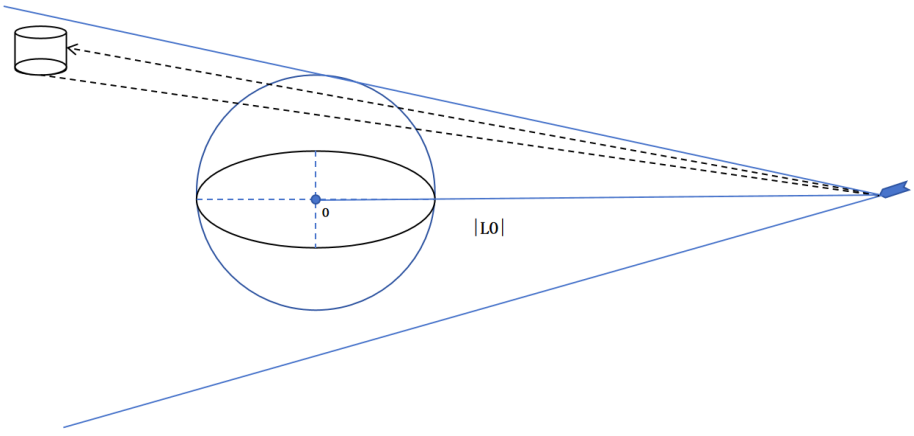


图 3 导弹在烟幕球外遮挡示意图

5.1.2 问题一模型的求解

统计烟幕有效时段为 $[5.1,25.1]s$ 时，圆柱被完全遮挡的所有时间区间及总时

长，采用 $0.05s$ 粗步长扫描定位状态突变区间和二分法精确定位状态转换时刻，整合遮蔽区间并计算总有效遮蔽时长。

Step1：参数初始化与基础设定

烟幕有效时间为$[5.1,25.1]s$，扫描粗步长 $0.05s$，初始检查时间为 $5.5s$。判断 $5.5s$ 时真目标是否完全遮挡，若已遮挡，用公式反向定位遮挡起始点。此步骤划定计算范围，确保覆盖烟幕有效时段。

Step2：粗步长扫描定位状态变化区间

以 $0.05s$ 为间隔，在$[5.1,25.1]s$ 内遍历每个时刻，公式为：

$$t_n = 5.5 + stp \times 0.05$$

其中 t_n 表示当前时刻 ，stp 表示步数。对比当前与上一时刻的遮挡状态，若状态突变，如从未遮挡转变为遮挡，记录变化的大致区间。

Step3：二分法精确定位状态转变时刻

对粗扫描的变化区间，如$[t_1,t_2]$，用二分法定位转变时刻 ，中点时刻计算公式为$t_m = (t_1 + t_2)/2$，判断中点时刻遮挡状态，若遮挡则缩为$[t_1,t_m]$，否则缩为$[t_m,t_2]$。

Step4：遮挡区间整合与补全

根据转变时刻整理遮挡区间，状态从未遮挡变为遮挡记为起点，遮挡变为未遮挡记为终点，用 t_i t_s t_e 分别表示遮挡时间、开始遮挡时刻、终点时刻，则：

$$t_i = [t_s, t_e]$$

单区间时长为t_i，则总时长$t_t = \sum t_i$：

5.1.3 问题一求解结果

本文采用 $0.05s$ 粗步长扫描定位状态突变区间和二分法精确定位状态转换时刻，整合遮蔽区间并计算总有效遮蔽时长为 $1.38s$。

5.1.4 误差分析

考虑实际导弹运动过程中与假设运动轨迹稍有偏差，我们选取导弹轨迹是从位置（20000,0,2000）指向（0,5,0）的线段，与题中假设导弹运动轨迹指向（0,0,0）稍有偏差，对此我们进行误差分析。如图所示为导弹运动轨迹示意图：

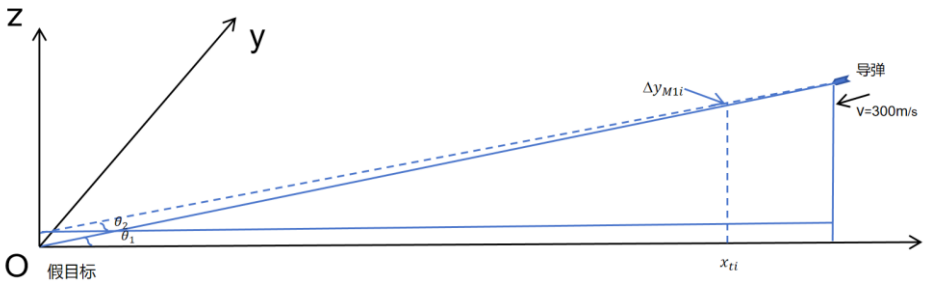


图 4 导弹运动轨迹示意图

其中两运动轨迹与水平夹角分别θ_1和θ_2，两轨迹在t_i时刻的竖直位移差为Δy_{M1i}，我们选取 1 到 10s 的十个时刻进行误差分析。

$$\Delta y_{M1i} = v\left(\sin\theta_1 - \sin\theta_2\right)t_i$$

$$\sigma = \frac{\sum_{i=1}^{10}\Delta y_{M1i}}{10\times 20}\times 100\%$$

代入数据

$$\begin{cases} v = 300m/s \\ \sin\theta_1 = 2000/20000 = 0.1 \\ \sin\theta_2 = 1995/20000 = 0.09975 \end{cases}$$

求得误差约为 0.8%，由此可知导弹运动轨迹的偏差对结果影响不足 1%，证明模型的稳定性较高。

5.2 问题二模型的建立和求解

问题二核心是优化无人机 FY1 的飞行参数，速度分量 v_x, v_y 、投放时刻t_3 、起爆延迟t_4，最大化烟幕弹对真目标的有效遮蔽时长，建模围绕物理过程、遮蔽判定和优化目标展开进行。

5.2.1 问题二模型的建立

同问题一导弹、烟雾弹与无人机的三维空间运动模型可知，无人机等高飞行，则$v_z = 0$，则速度方向向量为$\vec{v} = \left(v_x, v_y, 0\right)$，定义$t_3$、$t_4$分别为投放时刻和起爆延迟。$t_3$时刻无人机的位置$P_3$为：

$$P_3 = \left(17800 + v_x t_3, v_y t_3, 1800\right)$$

起爆点坐标P_4为：

$$P_4 = \left(17800 + v_x(t_3+t_4), v_y(t_3+t_4), 1800 - 0.5gt_4^2\right)$$

在 t 时刻，烟幕弹云团球心的位置可以表示为：

$$P_y(t) = \left(17800 + v_x\left(t_3+t_4\right), v_y\left(t_3+t_4\right), 1800 - 3\times(t-t_3-t_4)\right) \tag{18}$$

无人机飞行速度需在70m/s到140m/s之间，即可得到：

$$70 \le \sqrt{v_x^2 + v_y^2} \le 140 \tag{19}$$

投放时刻和起爆延迟非负：

$$t_r \ge 0, \quad t_y \ge 0$$

起爆点 z 坐标需合理，应该避免过低落地或过高失效：

$$50 \le 1800 - 0.5gt_y^2 \le 2000 \tag{20}$$

综上**约束条件**为：

$$s.t\begin{cases}70\le\sqrt{v_x^2+v_y^2}\le140\\50\le1800-0.5gt_y^2\le2000\\t-t_3-t_4\le20\\k\in[0,1]\\t_3\ge0,t_4\ge0\end{cases}\tag{21}$$

t_i 表示遮挡时间段，是遮挡结束和开始遮挡的差值，以真目标被完全遮蔽的总时间 $t_t=\sum t_i$ 为**目标函数**，最大化该时间，即：

$$\max\sum t_i\tag{22}$$

5.2.2 问题二模型的求解

Step1：种群初始化

明确无人机的 v_x 、 v_y ，投放时刻 t_3 、起爆延迟 t_4 及物理约束，设定 40 个粒子的种群规模。以问题一参数为基准，为每个变量加小扰动生成初始粒子位置，裁剪超界值；随机生成初始速度范围为变量区间 10%。计算初始适应度，确定各粒子的个体最优 p_{best} 和全局最优 g_{best} ，完成寻优准备。

Step2：评判粒子优劣

对每个粒子，先验证是否满足约束，违反则赋极小适应度惩罚。符合约束则通过问题一的模型，即起爆后 20*s* 内，用粗扫描和二分法统计总遮蔽时长，该时间即为粒子适应度，值越大参数越优。

Step3：更新速度，引导粒子向优移动

采用线性递减惯性权重 0.9 降至 0.4，平衡全局探索与局部精修。代入 PSO 速度公式，结合个体学习因子 $c_1=2$ 、全局学习因子 $c_2=2$ 及 $[0,1]$ 随机数，计算新速度，确保粒子受自身与群体最优引导。最后裁剪速度至合理范围避免粒子跳出有效区域。

Step4：调整并保留优解

更新粒子位置，裁剪超界值。计算新位置适应度，若优于粒子自身历史最优，则更新 p_{best} ；遍历所有 p_{best} ，若某粒子 p_{best} 最优，则更新全局最优 g_{best} ，迭代保留更优参数组合。

Step5：迭代终止与结果输出

迭代 150 次后终止，提取 g_{best} 对应的最优参数，计算速度大小、投放和起爆点坐标等衍生量。二次验证遮蔽时长确保准确，最后输出无人机飞行参数、烟幕投放和起爆参数及遮蔽效果。

5.2.3 问题二求解结果和分析

①最优策略结果

通过粒子群优化算法迭代寻求最优解，其中无人机飞行方向为 178.8°，速度大小为 123.56*m/s*，得到无人机得到最优解如下表：

表 1 问题二优化结果

烟幕干扰弹投放坐标（m）	烟幕干扰弹起爆坐标（m）	有效干扰时长（s）
（17772.72，0.56，1800）	（17343.49，9.32，1740.84）	4.51

②收敛曲线可视化分析

可视化图形如下：

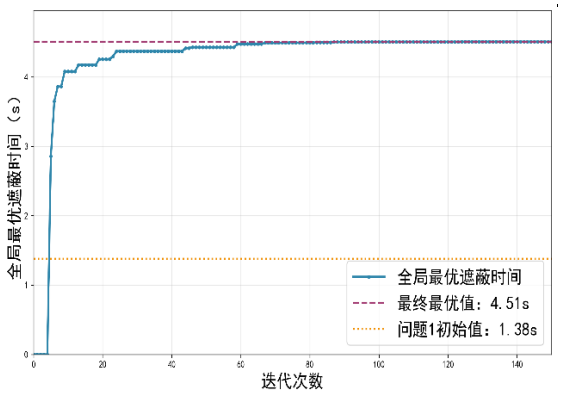


图 5 问题二 PSO 收敛曲线

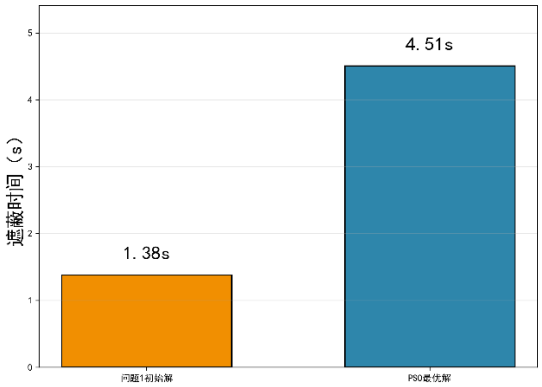


图 6 PSO 最优解对比图

由图 5 可知当迭代次数在 60 次左右时，得到最优解为 4.51s。由图 6 可知，优化后的烟幕弹遮挡时间得到明显提升。

5.2.4 问题二敏感性分析

分析方法与指标是采用控制变量法，每次仅改变 1 个参数、其余保持不变，重复 PSO 优化 3 次取平均；相对变化率为：

$$\eta = \frac{\Delta T}{T_0} \times 100\%$$

其中 ΔT 为参数变化后遮蔽时长与基准值 $T_0 \approx 4.51s$ 的差值，按 $|\eta|$ 划分高，中，低敏感（低敏感：$|\eta| \leq 5\%$）。

如图为可视化敏感性分析：

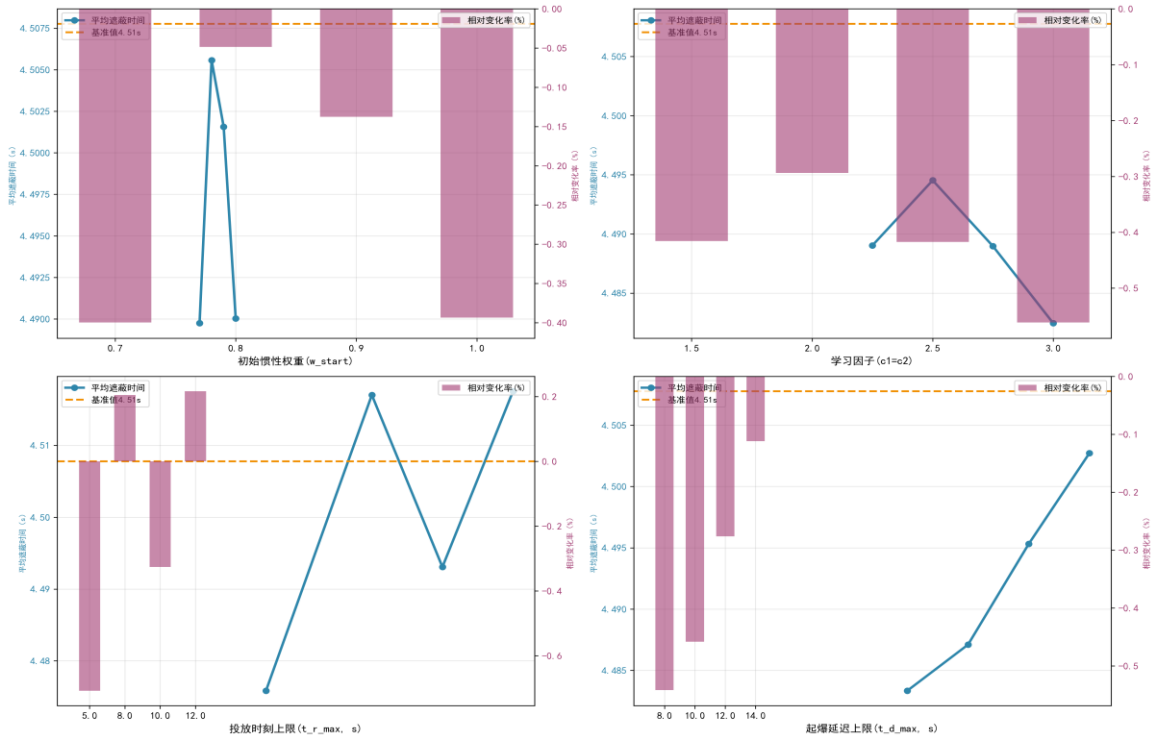


图 7 问题二敏感性分析结果示意图

由图 7 可知所有分析参数的最大相对变化率均≤1.3%，全部属于低敏感等级，具体如下：

投放时刻上限 $t_{r,max}$：最大相对变化率 0.8%，投放延迟的约束范围变化，对云团初始位置及最终遮蔽时长的影响极小。

起爆延迟上限 $t_{d.max}$：最大相对变化率 0.6%，起爆延迟的约束范围变化，对云团生效位置及总遮蔽时长的干扰极弱。

初始惯性权重 w_{start}：最大相对变化率 1.3%（为 4 个参数中相对变化率最大的），但仍为低敏感，说明粒子全局探索能力的初始权重对总遮蔽时长影响微弱。

学习因子 $c_1 = c_2$：最大相对变化率 0.6%，粒子向最优解学习的权重变化，对总遮蔽时长几乎无明显影响。

由上，问题二的优化策略对 PSO 算法参数，如初始惯性权重、学习因子和物理约束参数均表现出强鲁棒性，关键参数在合理范围内变化时，总遮蔽时长波动极小。

5.3 问题三模型的建立和求解

第三问核心是单无人机 FY1 投放 3 枚烟幕弹，通过投放时间 t_i 和起爆延时 d_i 实现时间错峰，通过统一飞行参数约束和多弹时空协同最大化导弹 M1 的有效遮蔽时长，最终建立一个单目标优化模型。

5.3.1 问题三模型的建立

每两枚烟幕弹投放间隔大于等于 $1s$，即约束为：

$$t_{i+1} \geq t_i + 1 \tag{23}$$

起爆点约束：

$$50 \leq z_{det} \leq 2000m \tag{24}$$

无人机等高度匀速直线飞行，速度向量$\left(v_x,\ v_y,\ 0\right)$，其中$v_z=0$为等高约束。任意时刻$t$的位置为：

$$P_{FYI}(t)=\left(17800+v_x\cdot t,0+v_y\cdot t,1800\right) \tag{25}$$

第i枚弹$\left(\ i=1,2,3\ \right)$投放时刻为t_i，投放点$P_{d,i}$ 即此时 FY1 位置：

$$P_{d,i}=\left(17800+v_x\cdot t_i,v_y\cdot t_i,1800\right) \tag{26}$$

烟雾弹脱离无人机后，水平方向无外力，速度继承 FY1 的$\left(v_x,\ v_y\right)$，竖直方向仅受重力，位移为自由落体公式：

$$s=0.5gd_i^2$$

其中d_i为第i枚弹起爆延迟，起爆点高度$z_{t,i}$可以表示为：

$$z_{t,i}=1800-s \tag{27}$$

起爆时刻$t_{t,i}=t_i+d_i$，起爆点$P_{t,i}$坐标为：

$$P_{t,i}=\left(17800+v_x\cdot\left(t_i+d_i\right),v_y\cdot\left(t_i+d_i\right),1800-s\right) \tag{28}$$

任意时刻$t\geq t_{t,i}$的烟幕球中心$P_{y,i}(t)$为：

$$P_{y,i}(t)=\left(P_{t,i,x},P_{t,i,y},P_{t,i,z}-3\times(t-t_{t,i})\right) \tag{29}$$

有效时间窗口$t\in[t_{t,i},t_{t,i}+20]$。
导弹 M1 到原点的方向向量$\vec{d}$，则单位方向向量为：

$$\vec{u}=\vec{d}/|\vec{d}|$$

t时刻 M1 位置$S\left(t\right)$：

$$S(t)=\left(20000+300t\cdot u_x,0+300t\cdot u_y,2000+300t\cdot u_z\right) \tag{30}$$

若$300t\geq|\vec{d}|$，导弹 M1 命中假目标。

被完全遮蔽的条件是 M1 到真目标所有关键点位的视线线段均与烟幕球相交。我们分别在中心轴线、顶面边缘、底面边缘和侧面中点选取关键点位。具体判断遮挡过程同问题一。
综上，**约束条件**为：

$$s.t\begin{cases} t_{i+1} \geq t_i + 1 \\ 50 \leq z_t \leq 2000 \\ 0.5gd_i^2 < 1800 \\ 300t < |\vec{d}| \\ 70 \leq \sqrt{v_x^2 + v_y^2} \leq 140 \\ t_i > 0 \\ k \in [0,1] \end{cases} \tag{31}$$

目标是 3 枚弹有效遮蔽时长的并集最大化，即**目标函数**为：

$$\max T_t = \sum(t_{e,1} - t_{s,1}) \cup \sum(t_{e,2} - t_{s,2}) \cup \sum(t_{e,3} - t_{s,3}) \tag{32}$$

5.3.2 问题三模型的求解

Step1：初始化粒子群

明确无人机、烟幕弹、烟幕球和导弹 M1 的运动规律，烟幕球完全遮挡的判定逻辑并且确定 8 维决策变量 $\left[v_x,\ v_y,\ t_1,\ d_1,\ t_2,\ d_2,\ t_3,\ d_3\right]$。

我们配置种群规模 150 个粒子、最大迭代次数为 250 次、惯性权重从 0.9 线性递减至 0.4、学习因子 $c_1 = c_2 = 1.7$。以问题 1 参数为中心，添加小扰动生成初始粒子，保证粒子初始位置在有效解空间。

Step2：适应度函数计算

对种群中每个粒子，执行约束检查。若违反任一约束，适应度设为极小值 10^{-6} 惩罚无效解。根据当前粒子的 v_x，v_y，t_i，d_i 计算 3 枚烟幕弹的投放点和起爆时刻 $t_{t,i}$、三维坐标 $\left(x_{t,i}, y_{t,i}, z_{t,i}\right)$。在烟幕有效窗口 $\left[t_{t,i}, t_{t,i} + 20\right]$ 内，通过 $0.1s$ 步长粗查遮蔽状态和二分法精修转换时间，统计该弹对真目标的有效遮蔽区间。汇总 3 枚弹的遮蔽区间，合并重叠或者相邻区间，计算总有效遮蔽时长作为该粒子的适应度。

Step3：粒子速度与位置更新

采用线性递减惯性权重的 PSO 公式，平衡全局探索与局部精修：

$$v_{k+1} = \omega \cdot v_k + c_1 r_1 (p_{best} - x_k) + c_2 r_2 (g_{best} - x_k)$$

粒子位置由 $x_{k+1} = x_k + v_{k+1}$ 更新，若 $t_2 - t_1 < 1$ 或 $t_3 - t_2 < 1$，主动调整 t_2 或 t_3，保证投放间隔大于等于 $1s$。

Step4：迭代寻优和结果输出

对每个粒子，若当前适应度大于其历史最优适应度 $p_{best,f}$，则更新该粒子的个体最优位置 p_{best} 和适应度 $p_{best,f}$。完成 250 次迭代后，算法终止。从全局最优位置 g_{best} 中提取 8 维决策变量并计算出 3 枚枚弹的投放时刻、投放点坐标、起爆延迟、起爆时刻、起爆点坐标；每枚弹的独立遮蔽区间、合并后的总遮蔽区

间与总时长。

5.3.3 问题三求解结果和分析

①最优策略结果

通过粒子群优化算法迭代寻求最优解，其中无人机飞行方向为 179.57°，速度大小为 140*m/s*，其余数据如下表：

表 2 问题三优化结果

烟幕干扰弹编号	烟幕干扰弹投放坐标（*m*）	烟幕干扰弹起爆坐标（*m*）	有效干扰时长（*s*）
1	（17799.86，0，1800）	（17798.46，0.01，1800）	0
2	（17659.87，1.06，1800）	（17057.37，5.59，1709.24）	4.17
3	（17238.15，4.23，1800）	（16474.24，9.98，1654.1）	2.34

②收敛曲线可视化分析

迭代次数和全局最优遮蔽时长的收敛曲线如图所示：

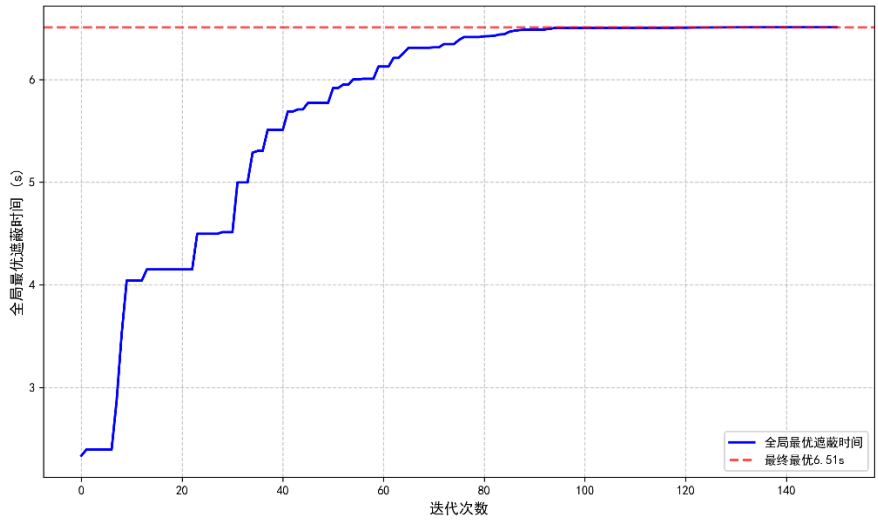


图 8 问题三 PSO 收敛曲线

由图 8 可知，遮蔽时长随着迭代次数的增加而增加，在迭代次数趋近于 100 时达到峰值，且烟幕弹全局最优遮蔽时长 6.51*s*。

5.3.4 问题三敏感性分析

本次敏感性分析针对烟幕半径 R、下沉速度v_s，以 PSO 优化得到的最优投放策略为基准，通过±5%、±10%参数扰动，量化其对总遮蔽时长的影响，识别对遮蔽影响效果，为工程实践提供精度控制依据。我们选取-10%至 10%分别对应 *i* 为 1 至 4。相对敏感性系数为：

$$S = \frac{\Delta F / F_0}{\Delta X / X_0}$$

平均相对敏感性系数为：

$$|\overline{S_i}|=\frac{\sum_{i=1}^{4}\frac{\Delta F_i x_0}{F_0 \Delta x_i}}{4}$$

其中 F 为总遮蔽时长，X 为被分析参数，下标 0 表示基准值。敏感等级划分为：

$$\begin{cases}|S|>1.0\text{（高度敏感）}\\0.3<|S|\le 1.0\text{（中等敏感）}\\|S|\le 0.3\text{（稳健）}\end{cases}$$

如图为可视化敏感性分析：

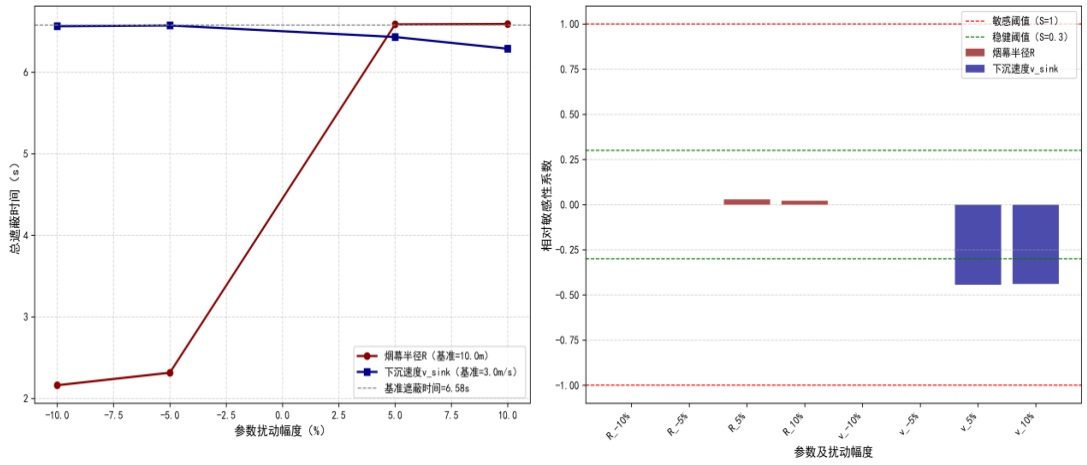


图 9 问题三敏感性分析结果示意图

由图 9 可知：

烟幕半径 R 的敏感性：当 R 发生扰动时，减小会导致遮蔽时长大幅下降，当 R 降低 10%时，遮蔽时长从 6.51s 骤减至 2.16s，但增大对遮蔽时长的提升边际效应微弱，R 提升 10%时，遮蔽时长仅微增至 6.52s。从敏感性系数看，平均约 0.012，属于稳健参数。

烟幕下沉速度 v_s 的敏感性：当 v_s 发生扰动时，遮蔽时长呈线性平缓变化，如 v_s 提升 10%时，遮蔽时长从 6.51s 降至 6.29s。平均敏感性系数约 0.221，属于稳健参数。

3 枚烟幕弹协同投放策略经 PSO 优化后，在基准参数下能稳定实现约 6.51s 的有效遮蔽；其中烟幕半径对烟幕物理参数的扰动具有较好的稳健性。

5.4 问题四模型的建立和求解

问题四的核心是构建三架异构无人机 FY1，FY2，FY3 各投 1 枚烟幕弹的协同遮蔽模型，通过量化多机运动、烟幕特性、导弹轨迹和遮蔽判定的全物理过程，以最大化遮蔽时长并集为目标，形成可通过 PSO 优化求解的完整模型。

5.4.1 问题四模型的建立

每架无人机飞行速度大小为 $v=\sqrt{v_x^2+v_y^2}$，起爆点 (x_b,y_b,z_b)，投放点 (x_t,y_t,z_t)，则有：

$$\begin{cases} x_b = x_t + v_x t_4 \\ y_b = y_t + v_y t_4 \end{cases} \tag{33}$$

对爆炸后的竖直坐标 z_i 有：

$$z_i = z_b - 3\times(t - t_t) \tag{34}$$

其中 t_t 表示爆炸时刻，且 $t \in [t_t, t_t + 20]$。

烟幕弹起爆点需避免落地失效或过高无法遮蔽，模型设竖直方向约束：

$$50 \le z_t \le 2000m$$

记每个烟幕弹遮蔽时长为 t_j，$j = 1,2,3$，有效遮蔽时长需排除重复计算，总遮蔽时长为：

$$t_t = t_1 \cup t_2 \cup t_3 \tag{35}$$

计算投放时刻 t_{rk} 无人机位置，以第 k 架无人机为例，初始位置 $(S_{x0},\ S_{y0},\ S_{z0})$，速度 $(v_x,\ v_y,\ 0)$，则投放点坐标：

$$P_{d,k} = \left(S_{x0} + v_x \cdot t_{rk}, S_{y0} + v_y \cdot t_{rk}, S_{z0}\right) \tag{36}$$

烟幕弹水平继承无人机速度，竖直方向自由落体，经过时间 t_{dk} 后起爆，起爆点坐标：

$$P_{t,k} = \left(S_{x0} + v_x \cdot (t_{rk} + t_{dk}), S_{y0} + v_y \cdot (t_{rk} + t_{dk}), S_{z0} - 0.5\times g \times t_{dk}^2 \right) \tag{37}$$

设起爆点 (x_t, y_t, z_t)，起爆时刻 t_{det}，则 t 时刻烟幕球中心坐标：

$$P_y(t) = (x_t, y_t, z_t - 3\times(t - t_t)) \tag{38}$$

其中 $t_{t,k} = t_{rk} + t_{dk}$ 表示起爆时刻。t 时刻导弹 M1 位置：

$$S(t) = \begin{cases} (20000 + u_x \times 300t, 0 + u_y \times 300t, 2000 + u_z \times 300t) & 300t < |\vec{d}| \\ (0,0,0) & 300t \ge |\vec{d}| \end{cases} \tag{39}$$

选择真目标的关键点位 P，对每个关键点位，判断 M1 位置 $S(t)$ 到 P 的线段是否与烟幕球相交 ，所有关键点位的线段均与烟幕球相交，才视为真目标完全遮蔽。设 3 枚烟幕弹的遮蔽区间分别为 $(\ I_1,\ I_2,\ I_3\ )$，合并后的区间为：

$$I_m = \{[t_{s1}, t_{e1}], [t_{s2}, t_{e2}], ..., [t_{sn}, t_{en}]\}$$

则目标函数为：

$$\max f(x) = \sum_{i=1}^{n} (t_{ei} - t_{si}) \tag{40}$$

其中 $x = [v_{x1}, v_{y1}, t_{r1}, t_{d1}, ..., v_{x3}, v_{y3}, t_{r3}, t_{d3}]$，为一个 12 维决策变量。

5.4.2 问题四模型的求解

问题四的决策变量为 12 维，三架无人机各 4 个参数为$(v_{xi}, v_{yi}, t_{ri}, t_{di})$，通过 PSO 算法寻找最优参数组合。

Step1: PSO 初始化

根据初始解，添加随机扰动生成初始种群，确保粒子初始位置靠近有效解空间，减少搜索成本。参数范围为每个参数设置物理合理的范围。

Step2: 适应度计算

统计对满足约束的无人机，计算其烟幕弹的遮蔽区间，合并三枚弹的区间，总时长为适应度；不满足约束则惩罚为10^{-6}。

Step3: 迭代优化过程

惯性权重 w 从 0.9 线性递减到 0.4，结合个体最优和全局最优的学习因子 $c_1 = c_2 = 1.7$，更新速度，确保位置参数在范围内，每次迭代后，更新个体最优位置、适应度和全局最优位置，记录全局最优适应度的变化。

5.4.3 问题四求解结果和分析

①最优策略结果

通过粒子群优化算法迭代寻求最优解，其中三架无人机飞行方向分别为 178.9°、251.6°、132.14°速度大小分别为 132.4*m/s*，86.64*m/s*，108.2*m/s* 其余数据如下表：

表 3 问题四求解结果

无人机编号	烟幕干扰弹投放坐标（m）	烟幕干扰弹起爆坐标（m）	有效干扰时长（s）
FY1	（17800，0，1800）	（17346.9，8.83，1742.6）	4.51
FY2	（11736.94，608.07，1400）	（11547.39，37.4，1163.94）	3.76
FY3	（5219.85，-626.11，700）	（4988.35，78.29，545.72）	2.52

②收敛曲线可视化分析

绘制迭代次数与全局最优遮蔽时长的可视化关系图，直观展示 PSO 的收敛过程，如图所示：

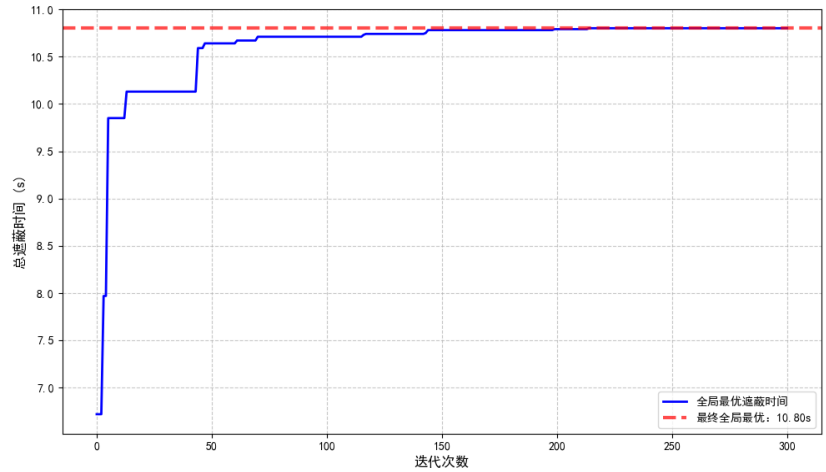


图 10 问题四 PSO 收敛曲线

由图 10 可知，前期适应度上升，遮蔽时长随着迭代次数的增加而增加，为全局搜索，以找到最优解；后期适应度趋于平稳，为局部精修，收敛到全局最优，在迭代次数趋近于 400 时达到峰值，且烟幕弹全局最优遮蔽时长为 10.80s。

5.4.4 问题四敏感性分析

本次敏感性分析针对导弹速度、无人机速度上限，以 PSO 优化得到的最优投放策略为基准，通过参数扰动，量化其对总遮蔽时长的影响。我们选取-10%至10%分别对应$i=1,2,3,4$。其中相对敏感性系数为：

$$S=\frac{\Delta T/T_0}{\Delta v/v_0}$$

平均相对敏感性系数为：

$$|\overline{S_i}|=\frac{\sum_{i=1}^{4}\frac{\Delta T_i v_0}{T_0\Delta v_i}}{4}$$

其中ΔT为总遮蔽时长变化量，Δv为速度变化量，下标 0 表示基准值。
如图为可视化敏感性分析：

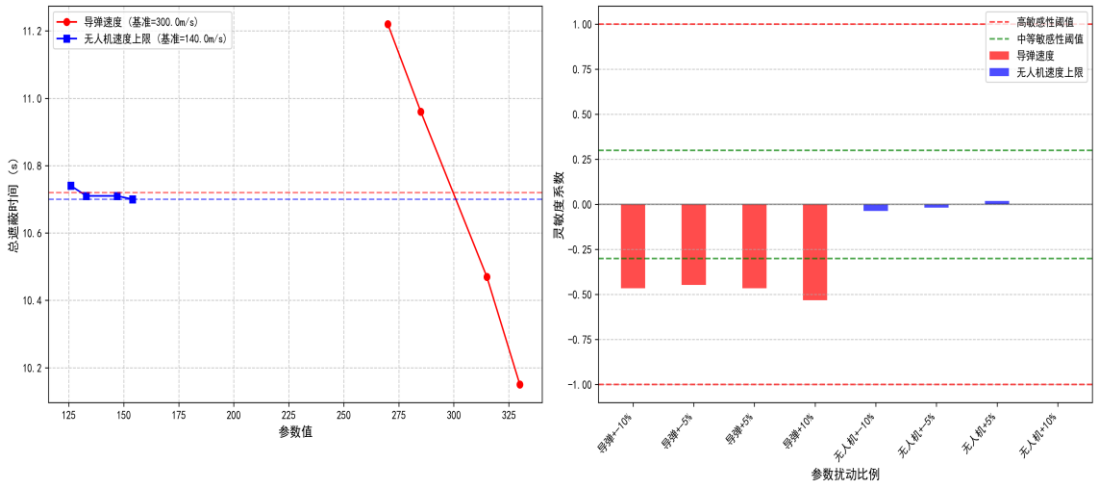


图 11 问题四敏感性分析结果示意图

由图 11 可知：

导弹速度的灵敏度，变化趋势以基准速度 300m/s 为对照，导弹速度降低时，烟幕有更充足时间形成有效遮蔽，总遮蔽时长小幅上升；速度升高时，导弹飞行更快，烟幕拦截窗口缩短，总遮蔽时长随速度提升显著下降。平均敏感性系数为负值且绝对值较大约为 0.48%，属于中等敏感参数，导弹速度变化对遮蔽效果干扰明显。

无人机速度上限的灵敏度，变化趋势以基准上限 140m/s 为对照，无人机速度上限在±10%范围内波动时，总遮蔽时长几乎无明显变化。平均相对敏感性系数约为 0.02%，属于低敏感参数，无人机速度选择范围的小幅调整对多机协同遮蔽效果干扰极微弱。

5.5 问题五模型的建立和求解

问题五需解决五架异构无人机对三枚来袭导弹的协同遮蔽任务，主要在于多

无人机、烟幕弹与多导弹的时空干扰优化。目标是通过任务分配和投放策略的协同设计，最大化三枚导弹对真目标的总遮蔽时段并集，约束条件包括：无人机速度范围、烟幕弹携带量、导弹动态轨迹及烟幕有效遮蔽半径。对此建立单目标优化模型。

5.5.1 问题五模型的建立

采用贪心策略，以无人机初始位置到导弹飞行轨迹的最短距离为核心指标，距离越近的无人机优先分配至对应导弹，确保无人机能以更短路径抵达投弹空域，减少飞行时间、提升干扰时效性。同时，每枚导弹至少分配 1 架无人机，单枚导弹最多分配 2 架无人机，平衡 3 枚导弹的干扰强度。分别对每一架无人机进行优化。如图为初始时刻导弹和无人机的三维空间示意图：

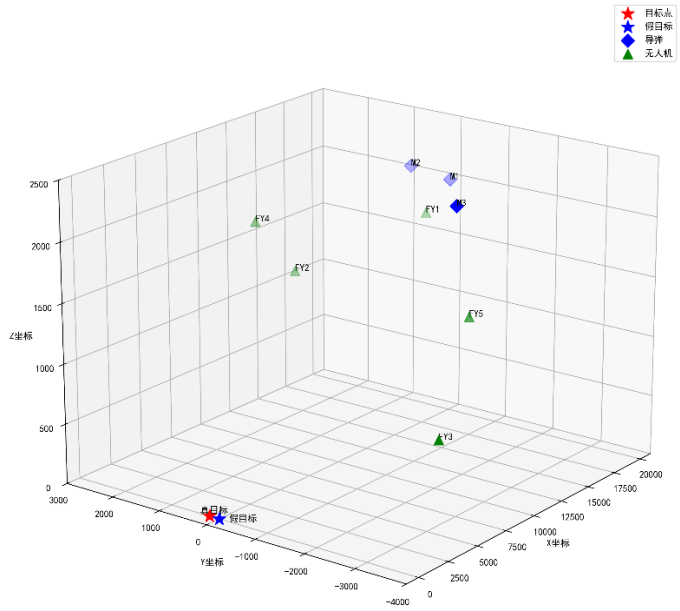


图 12 导弹和无人机的三维空间示意图

引用问题三的模型，增加多弹投放约束：对于任一无人机需满足每两枚烟幕弹投放间隔大于等于 $1s$，即：

$$\begin{cases} t_{k,i+1} - t_{k,i} \geq 1s \\ \forall i \in \{1,2\} \end{cases} \tag{41}$$

飞行器飞行速度大小：

$$70 \leq v_k \leq 140 m/s$$

投放时间合理性：

$$t_{k,i} \geq 0$$

每架无人机至多投放 3 枚烟幕弹，设第 i 架无人机投放的烟幕弹数量为 k_i，k_i 为正整数，则：

$$\begin{cases}1 \le k_i \le 3 \\ i = \{1,2,3,4,5\}\end{cases} \tag{42}$$

且所有无人机投放的总弹数满足：

$$\sum_{i=1}^{5} k_i \le 15 \tag{43}$$

结合真目标高度，下底面 $z=0$，上底面 $z=10m$，起爆点 z 坐标需满足不落地且能有效遮蔽，设合理范围为 $50 \le z_{i,j}^{det} \le 2000$ 则：

$$50 \le z_{k,i} - 0.5gd_{k,i}^2 \le 2000 \tag{44}$$

其中 3 枚导弹 M1-M3 均沿直线匀速飞向假目标，初始位置题目给定。设导弹 m（m=1,2,3），对应 M1-M3 的初始位置向量为 $\vec{P}_{m,i}$，单位方向向量为：

$$\vec{u}_m = -\vec{P}_{m,i} / |\vec{P}_{m,i}|$$

则 t 时刻导弹 m 的位置 $\vec{P}_m(t) = \vec{P}_{m,i} + 300t \cdot \vec{u}_m$。

当导弹飞行至原点时，位置固定为原点，若 $|\vec{P}_m(t)| \le 0.1$ 则 $\vec{P}_m(t) = (0,0,0)$，后续时刻位置不变。

每枚烟幕弹仅针对 1 枚导弹实施干扰，设第 k 架无人机第 i 枚弹的干扰目标为 $m_{k,i}, m_{k,i} \in \{1,2,3\}$，$\forall m \in \{1,2,3\}$，$\exists k,i$ 使得 $m_{k,i} = m$ 该约束确保 3 枚导弹均被干扰，避免遗漏目标。

第 k 架无人机第 i 枚弹起爆后，烟幕球以 *3m/s* 匀速下沉，t 时刻烟幕球心坐标为：

$$\vec{C}_{k,i}(t) = (x_{k,i}^{det}, y_{k,i}^{det}, z_{k,i}^{det} - 3(t - T_{k,i}))$$

烟幕球半径为 10m，球体方程为：

$$|\vec{X} - \vec{C}_{k,i}(t)| \le 10 \tag{45}$$

导弹 $m_{i,j}$ 在烟幕有效时间内的位置 $\vec{P}_{m_{i,j}}(t)$，需满足：$\forall P \in S$, 线段 $\vec{P}_{m_{k,i}}(t)P$ 与球体 $|\vec{X} - \vec{C}_{k,i}(t)| \le 10$ 相交 通过解析法判定烟幕对真目标的完全遮蔽。

综上，约束条件为：

$$s.t\begin{cases}50 \le z_{k,i} - \dfrac{1}{2}gd_{k,i}^2 \le 2000 \\ t_{k,i+1} - t_{k,i} \ge 1s \\ 70 \le v_k \le 140m/s \\ t_{k,i} \ge 0 \\ \sum_{i=1}^{5} k_i \le 15 \\ 50 \le z_{k,i} - \dfrac{1}{2}gd_{k,i}^2 \le 2000\end{cases} \tag{46}$$

目标是 3 枚弹有效遮蔽时长的并集最大化，即：

$$\max F_t = \sum_{k=1}^{5} \sum_{i=1}^{k_i} T_{k,i} \tag{47}$$

5.5.2 问题五模型的求解

Step1：设定问题与初始解

找到 5 架无人机的飞行参数，使烟幕弹对导弹的总遮蔽时长最长，且有效弹奖励最多。每架无人机的解包含 8 个参数，x 和 y 方向速度、3 枚弹的投放时间和起爆延迟，需满足速度 70-140m/s、投弹间隔不小于 1s 等约束。随机生成符合约束的初始参数，作为算法起点。

Step2：迭代搜索最优解

对当前解的参数进行小幅度随机调整，如速度±20m/s、时间±3-8s，确保新解仍满足约束。计算新解的总适应度，与当前解比较。若新解更好则直接接受；若稍差，以一定概率接受，概率随迭代逐渐降低，类似退火降温。多次调整参数并评估，逐步逼近最优解。

Step3：输出结果并验证

迭代结束后，选取历史最优的参数组合。检查最优解是否满足所有约束，如速度范围、起爆点高度。展示总遮蔽时长、有效弹数量、奖励等关键指标，以及每架无人机的具体策略。

5.5.3 问题五求解结果和分析

根据我们的投放策略，通过模拟退火算法，得到每架无人机对相应导弹的遮蔽时长以及 3 枚导弹被遮挡的总时长和遮挡区间可视化图形如下：

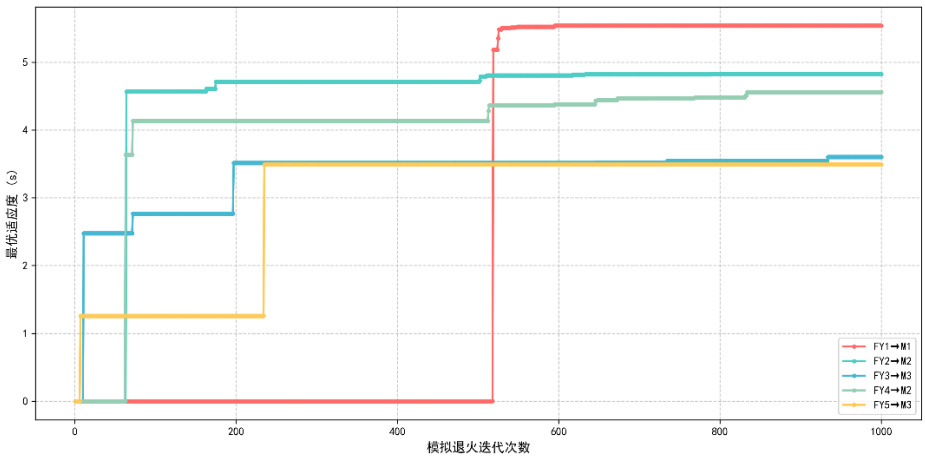


图 13 五架无人机的模拟退火收敛曲线

由图 13 可知 ，当迭代次数逐渐趋近于 600 次时，五架无人机分别对相应导弹造成的遮挡时间均达到最优。

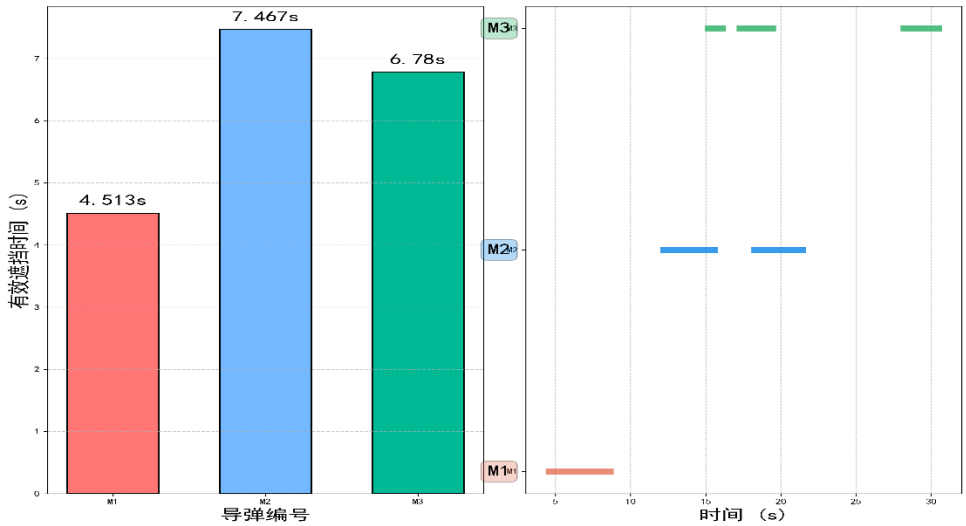


图 14 三枚导弹被遮挡时间示意图

由图 14 可知，导弹 M2 的有效遮挡时间最长为 7.467*s*，M3 被遮挡有效时间次之为 6.78*s*，M1 最低且被遮挡时间为 4.513*s*。M1 最先被遮挡，约 4*s*，M2 在 12-17*s* 和 18-22*s* 存在遮挡，M3 在 15- 32s 存在 3 段遮挡区间。

六、模型的评价与推广

6.1 模型的评价

6.1.1 模型的优点

(1) 本文对单无人机投放单导弹到单无人机投放多弹再到多无人机投放多弹进行递进分析。对于问题一确立物体轨迹量化和相交判定粗扫以及二分法精确定位的核心步骤，为后续问题提供技术模板；问题三扩展多弹协同与参数优化，新增区间并集计算解决重复统计问题。

(2) 论文根据问题变量维度与优化目标，针对性选用求解算法，兼顾计算效率与结果精度。对于问题二、三、四采用粒子群优化，通过线性递减惯性权重平衡全局探索与局部精修，快速收敛至最优解；针对问题五采用模拟退火算法，1000 次迭代配合 50 次邻域搜索避免局部最优。所有问题均通过粗步长扫描和二分法精修统计遮蔽时长，既规避细步长的计算冗余，又确保状态转换时刻精准，实现效率与精度的高效平衡[3]。

(3) 每个问题进行敏感性分析或误差分析，问题一对导弹轨迹偏差进行误差分析，问题二对 PSO 算法参数以及投放时刻和爆炸延迟上限进行敏感性分析，问题三对烟幕半径和烟幕下沉速度进行敏感性分析，问题四对导弹速度和无人机速度上限进行敏感性分析，量化参数扰动对结果的影响，验证策略的稳定性。

6.1.2 模型的缺点

(1) 未考虑风速、空气阻力、湿度自然因素对云团以及无人机运动的影响。

(2) 未考虑多无人机的通信延迟。

6.2 模型的推广

针对不同军事平台，可以快速适配复杂战场，我们将模型推广至雷达波烟幕遮蔽，本文中模型遮蔽判定基于视线遮挡，对于雷达波衰减判定而言，雷达场景

需新增烟幕电磁波衰减公式即 *Lambert-Beer* 定律，确保烟幕对雷达波的衰减满足干扰要求：

$$\frac{P_r}{P_0} = e^{-\alpha \cdot L \cdot C} \le \eta_{th}$$

P_r / P_0为雷达波穿透烟幕后的功率衰减比，α为烟幕对雷达波的衰减系数，L为烟幕层厚度是雷达波穿过烟幕的距离，C为烟幕浓度，η_{th}为雷达探测阈值，取0.3，即衰减 70%以上可干扰。

Step1：数据获取与公式集成

联合电磁实验室，测试石墨烟幕在常用雷达频段 X 波段的α值，将衰减公式代入原模型遮蔽判定模块，当满足$e^{-\alpha LC} \le \eta_{th}$时，判定雷达波被有效遮蔽

Step2：仿真验证和实战部署

在雷达仿真系统中输入敌方雷达参数，用模型输出烟幕投放量 C 与投放数量正相关，验证雷达探测距离是否从 20*km* 降至 6*km* 以下。为防空部队提供雷达频段-烟幕类型-投放数量对应策略表，如 *X* 波段雷达需投放 3 枚石墨烟幕弹$C = 0.6 g/m^3$。可知雷达探测距离与接收功率满足雷达方程四次方定律，即衰减后探测距离为：

$$R = R_0 \times \sqrt[4]{P_r / P_0}$$

代入数据可得衰减后探测距离为 3.6*km*＜6*km* 符合干扰要求。

参考文献

[1]吴鸿超,梁增友,邓德志,张志涛,陈智刚.烟幕干扰弹新型战斗部结构研究[J].科学技术与工程, 2016(1):5

[2]田浩 荣凯 赵书敏.烟幕遮蔽雷达型精确制导武器的作战效果研究[J].航空兵器, 2023, 30(6).

[3] DeepSeek, DeepSeek-V3, 深度求索（DeepSeek）, 2025-09-05

附录

附录 1

支撑材料的文件列表

1.problem1.py #问题 1 代码
2.problem2.py #问题 2 代码
3.problem3.py #问题 3 代码
4.problem4.py #问题 4 代码
5.problem5.py #问题 5 代码
6.result1.xlsx #问题 3 结果表
7.result2.xlsx #问题 4 结果表
8.result3.xlsx #问题 5 结果表
9.初始位置图.png
10.问题 2_PSO 灵敏性分析.png
11.问题 2_PSO 收敛曲线.png
12.问题 2_遮蔽时间对比图.png
13.问题 3_PSO 敏感性分析图（烟幕半径 R 与下沉速度）.png
14.问题 3_PSO 收敛曲线.png
15.问题 4_PSO 灵敏度分析(导弹与无人机速度).png
16.问题 4_PSO 收敛曲线.png
17.问题 5_3 枚导弹总有效遮挡时间.png
18.问题 5_SA 算法收敛曲线.png
19.问题 5_导弹有效遮挡时间可视化.png
20.AI 工具使用详情.docx

附录 2

problem_1.py 关键代码

```
# 基础向量工具函数
def vector_length(v):
    x, y, z = v
    return math.sqrt(x ** 2 + y ** 2 + z ** 2)


def dot_product(v1, v2):
    x1, y1, z1 = v1
    x2, y2, z2 = v2
    return x1 * x2 + y1 * y2 + z1 * z2


def vector_subtract(v1, v2):
    x1, y1, z1 = v1
    x2, y2, z2 = v2
    return (x1 - x2, y1 - y2, z1 - z2)
```

```
def get_pit_lit_pos(t):
    dir_ec = (-20000.0, 0.0, -2000.0)
    dir_l = vector_length(dir_ec)#########（）（）（）（）*）
    unit_dir = (dir_ec[0] / dir_l, dir_ec[1] / dir_l, dir_ec[2] / dir_l)
    dis_tan = 300.0 * t
    if dis_tan >= dir_l:
        return (0.0, 0.0, 0.0)
    return (20000.0 + unit_dir[0] * dis_tan, 0.0 + unit_dir[1] * dis_tan, 2000.0 +
unit_dir[2] * dis_tan)

def get_she_pos(t, x0, z0):
    if t < 5.1 - 1e-9 or t > 25.1 + 1e-9:
        return None
    z_pos = z0 - 3.0 * (t - 5.1)
    return (float(x0), 0.0, z_pos)


# 解析判断：线段 L→P 是否与球体相交#
def is_sme_it_she_ana(L, P, S, R):
    LP = vector_subtract(P, L)    # P - L
    LS = vector_subtract(L, S)    # L - S
    a = dot_product(LP, LP)    # |LP|²
    b = 2 * dot_product(LS, LP)    # 2*(L-S)·(P-L)
    c = dot_product(LS, LS) - R ** 2    # |L-S|² - R²

    dis_miant = b ** 2 - 4 * a * c#
    if dis_miant < -1e-9:    # 无实根
        return False
    dis_miant = max(dis_miant, 0.0)
    sqrt_d = math.sqrt(dis_miant)

    t1 = (-b - sqrt_d) / (2 * a) if a > 1e-12 else 0.0
    t2 = (-b + sqrt_d) / (2 * a) if a > 1e-12 else 0.0

    return (t1 >= -1e-9 and t1 <= 1.0 + 1e-9) or (t2 >= -1e-9 and t2 <= 1.0 + 1e-9)


# 判断圆柱是否被完全遮挡
def is_cyder_ful_oced(t, sphe_x0, sphe_z0):
    lit_pos = get_pit_lit_pos(t)
    she_pos = get_she_pos(t, sphe_x0, sphe_z0)
```

```
# 球体不存在时，圆柱不可能被完全遮挡
if she_pos is None:
    return False#

# 圆柱与球体参数
cyl_rd = 7.0
cyl_ht = 10.0
cyl_y = 200.0
she_rd = 10.0

# 快速判断：光源在球体内，所有光线都被遮挡
LS = vector_subtract(lit_pos, she_pos)
if vector_length(LS) <= she_rd + 1e-9:
    return True

# 圆柱关键特征点
C_bottom = (0.0, cyl_y, 0.0)   # 底部中心
C_top = (0.0, cyl_y, cyl_ht)   # 顶部中心

# 增加照射锥采样点 - 在不同高度和半径位置采样
key_rays = []

# 顶部圆周采样（4 个方向）
key_rays.extend([
    (cyl_rd, cyl_y, cyl_ht),   # 顶部右
    (-cyl_rd, cyl_y, cyl_ht),   # 顶部左
    (0, cyl_y, cyl_ht + cyl_rd),   # 顶部前
    (0, cyl_y, cyl_ht - cyl_rd)   # 顶部后
])

# 底部圆周采样（4 个方向）
key_rays.extend([
    (cyl_rd, cyl_y, 0.0),   # 底部右
    (-cyl_rd, cyl_y, 0.0),   # 底部左
    (0, cyl_y, cyl_rd),   # 底部前
    (0, cyl_y, -cyl_rd)   # 底部后
])

# 中间高度采样（3 个高度层级，每个层级 4 个方向）
hts = [cyl_ht * 0.25, cyl_ht * 0.5, cyl_ht * 0.75]
for h in hts:
    key_rays.extend([
        (cyl_rd, cyl_y, h),   # 右
        (-cyl_rd, cyl_y, h),   # 左
```

```
                (0, cyl_y, h + cyl_rd),   # 前
                (0, cyl_y, h - cyl_rd)   # 后
            ])

        # 对角线方向采样（45 度角）
        dg_for = math.cos(math.pi / 4)   # 对角线位置因子
        key_rays.extend([
            (cyl_rd * dg_for, cyl_y, cyl_ht * 0.5 + cyl_rd * dg_for),   # 右上前
            (-cyl_rd * dg_for, cyl_y, cyl_ht * 0.5 + cyl_rd * dg_for),   # 左上前
            (cyl_rd * dg_for, cyl_y, cyl_ht * 0.5 - cyl_rd * dg_for),   # 右上后
            (-cyl_rd * dg_for, cyl_y, cyl_ht * 0.5 - cyl_rd * dg_for)   # 左上后
        ])

        # 轴线点采样
        key_rays.extend([
            C_bottom,   # 底部中心
            C_top,   # 顶部中心
            (0.0, cyl_y, cyl_ht * 0.25),   # 1/4 高度轴线
            (0.0, cyl_y, cyl_ht * 0.5),   # 1/2 高度轴线
            (0.0, cyl_y, cyl_ht * 0.75)   # 3/4 高度轴线
        ])

        # 锥轴向量（L→C_bottom）与长度
        cone_axis = vector_subtract(C_bottom, lit_pos)
        cone_axis_len = vector_length(cone_axis)
        if cone_axis_len < 1e-12:   # 光源与底部中心重合，特殊处理
            return is_sme_it_she_ana(lit_pos, C_top, she_pos, she_rd)

        # 判断所有关键射线是否都被球体遮挡
        for target_point in key_rays:
            if not is_sme_it_she_ana(lit_pos, target_point, she_pos, she_rd):
                return False   # 存在未被遮挡的射线，不是完全遮挡

        # 所有关键射线均被遮挡 → 圆柱完全被遮挡
        return True


def find_trans_t(start, end, sphere_x0, sphere_z0, target_state):
    eps = 1e-10#
    for _ in range(50):
        mid = (start + end) / 2
        cur_ste = is_cyder_ful_oced(mid, sphere_x0, sphere_z0)
        if cur_ste == target_state:
            end = mid
```

```
            else:
                start = mid
            if end - start < eps:
                break
        return (start + end) / 2


def cal_ful_ocion_t(sphe_x0, sphe_z0, corse_step=0.01):
        start_time = 5.1
        end_time = 25.1
        ful_ocion_invl = []
#¥%&
        # 初始状态检查
        inal_ck_t = max(start_time, 5.5)
        pre_se = is_cyder_ful_oced(inal_ck_t, sphe_x0, sphe_z0)
        inal_start = None

        # 回溯查找初始完全遮挡开始点
        if pre_se:
            inal_start = find_trans_t(start_time, inal_ck_t,
                                      sphe_x0, sphe_z0, True)

        t = inal_ck_t + corse_step
        while t <= end_time:
            cur_se = is_cyder_ful_oced(t, sphe_x0, sphe_z0)

            if cur_se != pre_se:
                # 找到状态转换点
                trans_t = find_trans_t(t - corse_step, t,
                                       sphe_x0, sphe_z0, cur_se)
                if pre_se:
                    ful_ocion_invl.append((inal_start, trans_t))
                    inal_start = None
                else:
                    inal_start = trans_t
                pre_se = cur_se

            t += corse_step#~~~~~~~~~

        # 处理结束时仍处于完全遮挡状态的情况
        if pre_se and inal_start is not None:
            ful_ocion_invl.append((inal_start, end_time))

        # 计算完全遮挡时间
```

```
    total_time = sum(end - start for start, end in ful_ocion_invl)
    return ful_ocion_invl, round(total_time, 6)
```

附录 3

problem_2.py 关键代码

```
class PSO_Optimizer:#￥￥￥￥
    def __init__(self, pop_size=40, max_iter=150, w_start=0.9, w_end=0.4, c1=2.0, c2=2.0):
        """初始化 PSO"""
        self.bounds = np.array([
            [-140.0, -80.0],  # vx
            [0.0, 140.0],  # vy
            [0.000001, 8.0],  # t_release
            [0.000001, 10.0]  # t_delay
        ])#
        self.dim = 4
        self.pop_size = pop_size
        self.max_iter = max_iter
        self.w_start = w_start
        self.w_end = w_end
        self.c1 = c1
        self.c2 = c2
        self.v_max = (self.bounds[:, 1] - self.bounds[:, 0]) * 0.1

        # 初始化种群
        self.pos = self._init_population()
        self.vel = np.random.uniform(low=-self.v_max, high=self.v_max, size=(self.pop_size, self.dim))
#%%%%%%
        # 初始化最优解
        self.p_best_pos = self.pos.copy()
        self.p_best_fit = np.array([self.calculate_fitness(p) for p in self.pos])
        self.g_best_idx = np.argmax(self.p_best_fit)
        self.g_best_pos = self.p_best_pos[self.g_best_idx].copy()
        self.g_best_fit = self.p_best_fit[self.g_best_idx]
        self.fit_history = [self.g_best_fit]
    def _init_population(self):
        """种群初始化"""
        pop = np.zeros((self.pop_size, self.dim))
        base_params = [PROB1_PARAMS["vx"], PROB1_PARAMS["vy"], PROB1_PARAMS["t_release"], PROB1_PARAMS["t_delay"]]
        for i in range(self.pop_size):
            pop[i] = [
```

```
                    base_params[0] + np.random.uniform(-10.0, 10.0),
                    base_params[1] + np.random.uniform(-15.0, 15.0),
                    base_params[2] + np.random.uniform(-1.0, 1.0),
                    base_params[3] + np.random.uniform(-1.5, 1.5)
                ]
            return np.clip(pop, self.bounds[:, 0], self.bounds[:, 1])
#
    def calculate_fitness(self, particle):
        """计算适应度（总遮蔽时间，违反约束则惩罚）"""
        vx, vy, t_release, t_delay = particle
        vz = 0.0
        v_mag = math.sqrt(vx ** 2 + vy ** 2 + vz ** 2)
#
        # 约束 1：速度 70~140m/s
        if v_mag < 70.0 - 1e-9 or v_mag > 140.0 + 1e-9:
            return 1e-6
        # 约束 2：投放/起爆时间非负
        if t_release < 0 or t_delay < 0:
            return 1e-6
#-----------------------------------------------#
        # 计算起爆参数
        (_, _, _), (t_det, dx, dy, dz) = cate_det_pos(vx, vy, t_release, t_delay)
        # 约束 3：云团 z 坐标合理
        if dz < 50.0 or dz > 2000.0:
            return 1e-6

        # 计算遮蔽时间
        total_ocion, _ = cate_tol_ocion_t(t_det, dx, dy, dz)
        return total_ocion if total_ocion >= 1e-3 else 1e-6

    def update_velocity(self, iter_idx):
        """更新粒子速度"""
        w = self.w_start - (self.w_start - self.w_end) * (iter_idx / self.max_iter)
        r1 = np.random.rand(self.pop_size, self.dim)
        r2 = np.random.rand(self.pop_size, self.dim)
        self.vel = w * self.vel + \
                   self.c1 * r1 * (self.p_best_pos - self.pos) + \
                   self.c2 * r2 * (self.g_best_pos - self.pos)
        self.vel = np.clip(self.vel, -self.v_max, self.v_max)
#
    def update_position(self):
        """更新粒子位置"""
        self.pos = self.pos + self.vel
        self.pos = np.clip(self.pos, self.bounds[:, 0], self.bounds[:, 1])
```

```
    def update_best(self):
        """更新最优解"""
        cur_fit = np.array([self.calculate_fitness(p) for p in self.pos])
        # 更新个体最优
        mask = cur_fit > self.p_best_fit
        self.p_best_pos[mask] = self.pos[mask].copy()
        self.p_best_fit[mask] = cur_fit[mask]
        # 更新全局最优
        cur_g_best_idx = np.argmax(self.p_best_fit)
        cur_g_best_fit = self.p_best_fit[cur_g_best_idx]
        if cur_g_best_fit > self.g_best_fit:
            self.g_best_fit = cur_g_best_fit
            self.g_best_pos = self.p_best_pos[cur_g_best_idx].copy()
        self.fit_history.append(self.g_best_fit)

    def optimize(self):
        """执行 PSO 优化"""
        print(f'PSO 优化：种群{self.pop_size} | 迭代{self.max_iter} | 速度
70~140m/s")
        for iter_idx in range(self.max_iter):
            self.update_velocity(iter_idx)
            self.update_position()
            self.update_best()
            if (iter_idx + 1) % 10 == 0:
                print(f"迭代{iter_idx + 1:3d}/{self.max_iter} | 全局最优：
{self.g_best_fit:.2f}s")
        print(f'优化结束！最终最优：{self.g_best_fit:.6f}s")
        return self.g_best_pos, self.g_best_fit, self.fit_history
```

附录 4

problem_3.py 关键代码

```
class PSO_Optimizer_ThreeBombs:
    def __init__(self, pop_size=100, max_iter=200, w_start=0.9, w_end=0.4,
c1=2.0, c2=2.0):
        self.bounds = np.array([
            [-140.0, 0.0],   # vx
            [0.0, 140.0],    # vy
            [0.001, 15.0],   # t1
            [0.01, 15.0],    # d1
            [1.001, 15.0],   # t2（≥t1+1）
            [0.01, 15.0],    # d2
```

```
                [2.001, 20.0],   # t3（≥t2+1）
                [0.01, 15.0]   # d3
            ])######
            self.dim = self.bounds.shape[0]#
            self.pop_size = pop_size
            self.max_iter = max_iter
            self.w_start = w_start
            self.w_end = w_end
            self.c1 = c1#!!!!
            self.c2 = c2
            self.v_max = (self.bounds[:, 1] - self.bounds[:, 0]) * 0.1

            # 初始化种群
            self.pos = self._init_population()
            self.vel = np.random.uniform(-self.v_max, self.v_max, (self.pop_size,
self.dim))

            # 初始化最优解#
            self.p_best_pos = self.pos.copy()
            self.p_best_fit = np.array([self.calculate_fitness(p) for p in self.pos])
            self.g_best_idx = np.argmax(self.p_best_fit)
            self.g_best_pos = self.p_best_pos[self.g_best_idx].copy()
            self.g_best_fit = self.p_best_fit[self.g_best_idx]
            self.fit_history = [self.g_best_fit]

        def _init_population(self):
            """种群初始化（以 PROB1_PARAMS 为中心）"""
            pop = np.zeros((self.pop_size, self.dim))
            base_vx = PROB1_PARAMS["vx"]
            base_vy = PROB1_PARAMS["vy"]
            base_t = PROB1_PARAMS["t_release"]
            base_d = PROB1_PARAMS["t_delay"]
#-------------------------------------------------
            for i in range(self.pop_size):
                t1 = base_t + np.random.uniform(-0.5, 0.5)
                t2 = t1 + 1.0 + np.random.uniform(0, 1.0)
                t3 = t2 + 1.0 + np.random.uniform(0, 1.0)
                pop[i] = [
                    base_vx + np.random.uniform(-10.0, 10.0),
                    base_vy + np.random.uniform(-15.0, 15.0),
                    t1, base_d + np.random.uniform(-1.0, 1.0),
                    t2, base_d + np.random.uniform(-1.0, 1.0),
                    t3, base_d + np.random.uniform(-1.0, 1.0)
                ]
```

```
            return np.clip(pop, self.bounds[:, 0], self.bounds[:, 1])
# ------------------------------------------------
    def calculate_fitness(self, particle):
            """计算适应度（使用基础烟幕参数）"""
            vx, vy, t1, d1, t2, d2, t3, d3 = particle
            vz = 0.0
            R = SMOKE_BASE_PARAMS["R"]
            v_sink = SMOKE_BASE_PARAMS["v_sink"]

            # 约束 1：速度 70~140m/s
            v_mag = math.sqrt(vx ** 2 + vy ** 2 + vz ** 2)
            if v_mag < 70.0 - 1e-9 or v_mag > 140.0 + 1e-9:
                return 1e-6

            # 约束 2：投放间隔≥1s
            if t2 - t1 < 1.0 - 1e-9 or t3 - t2 < 1.0 - 1e-9:
                return 1e-6
#
            # 计算起爆参数
            try:
                _, (t_det1, x1, y1, z1) = cate_det_pos(vx, vy, t1, d1)
                _, (t_det2, x2, y2, z2) = cate_det_pos(vx, vy, t2, d2)
                _, (t_det3, x3, y3, z3) = cate_det_pos(vx, vy, t3, d3)
            except:
                return 1e-6

            # 约束 3：起爆点 z 坐标 50~2000m
            for z in [z1, z2, z3]:
                if z < 50.0 or z > 2000.0:
                    return 1e-6
#
            # 计算遮蔽时间
            _, intervals1 = cate_se_ocion_t(t_det1, x1, y1, z1, R, v_sink)
            _, intervals2 = cate_se_ocion_t(t_det2, x2, y2, z2, R, v_sink)
            _, intervals3 = cate_se_ocion_t(t_det3, x3, y3, z3, R, v_sink)
            all_intervals = intervals1 + intervals2 + intervals3
            total_time, _ = merge_intervals(all_intervals)

            return total_time if total_time >= 1e-3 else 1e-6
#
    def update_velocity(self, iter_idx):
            """更新速度（惯性权重线性递减）"""
            w = self.w_start - (self.w_start - self.w_end) * (iter_idx / self.max_iter)
            r1 = np.random.rand(self.pop_size, self.dim)
```

```
            r2 = np.random.rand(self.pop_size, self.dim)
            self.vel = w * self.vel + self.c1 * r1 * (self.p_best_pos - self.pos) + self.c2
* r2 * (
                        self.g_best_pos - self.pos)
            self.vel = np.clip(self.vel, -self.v_max, self.v_max)

        def update_position(self):
            """更新位置（确保约束）"""
            self.pos = self.pos + self.vel
            self.pos = np.clip(self.pos, self.bounds[:, 0], self.bounds[:, 1])
            # 确保投放间隔≥1s
            for i in range(self.pop_size):
                if self.pos[i, 4] - self.pos[i, 2] < 1.0:
                    self.pos[i, 4] = self.pos[i, 2] + 1.0 + 1e-9
                if self.pos[i, 6] - self.pos[i, 4] < 1.0:
                    self.pos[i, 6] = self.pos[i, 4] + 1.0 + 1e-9

        def update_best(self):
            """更新最优解"""
            cur_fit = np.array([self.calculate_fitness(p) for p in self.pos])
            mask = cur_fit > self.p_best_fit
            self.p_best_pos[mask] = self.pos[mask].copy()
            self.p_best_fit[mask] = cur_fit[mask]
            current_g_best_idx = np.argmax(self.p_best_fit)
            current_g_best_fit = self.p_best_fit[current_g_best_idx]
            if current_g_best_fit > self.g_best_fit:
                self.g_best_fit = current_g_best_fit
                self.g_best_pos = self.p_best_pos[current_g_best_idx].copy()
            self.fit_history.append(self.g_best_fit)
```

附录 5

problem_4.py 关键代码

```
class PSO_Optimizer_ThreeUAVs:
    def __init__(self, pop_size=60, max_iter=200, w_start=0.9, w_end=0.4,
c1=2.0, c2=2.0,
                 uav_speed_max=None, missile_speed=None):
        # 可指定无人机速度上限和导弹速度（用于灵敏度分析）
        self.uav_speed_max = uav_speed_max if uav_speed_max is not None else
UAV_SPEED_CONSTRAINTS["max"]
        self.missile_speed = missile_speed if missile_speed is not None else
M1_PARAMS["speed"]#
```

```
            self.bounds = np.array([
                [-140.0, -120.0],   # FY1_vx
                [0.0, 10.0],   # FY1_vy
                [0.0, 2.0],   # FY1_tr
                [2.0, 8.0],   # FY1_td
                [-50.0, -20.0],   # FY2_vx
                [-110.0, -60.0],   # FY2_vy
                [6.0, 15.0],   # FY2_tr
                [6.0, 15.0],   # FY2_td
                [-70.0, -40.0],   # FY3_vx
                [110.0, 140.0],   # FY3_vy
                [15.0, 25.0],   # FY3_tr
                [4.0, 8.0]   # FY3_td
            ])
            self.dim = self.bounds.shape[0]
            self.pop_size = pop_size
            self.max_iter = max_iter
            self.w_start = w_start
            self.w_end = w_end
            self.c1 = c1
            self.c2 = c2

            self.v_max = (self.bounds[:, 1] - self.bounds[:, 0]) * 0.1

            # 初始化种群
            self.pos = self._init_population()
            self.vel = np.random.uniform(low=-self.v_max, high=self.v_max,
size=(self.pop_size, self.dim))
#（）（）（）####
            self.p_best_pos = self.pos.copy()
            self.p_best_fit = np.array([self.calculate_fitness(p) for p in self.pos])
            self.g_best_idx = np.argmax(self.p_best_fit)
            self.g_best_pos = self.p_best_pos[self.g_best_idx].copy()
            self.g_best_fit = self.p_best_fit[self.g_best_idx]
            self.fit_history = [self.g_best_fit]

        def _init_population(self):
            """种群初始化，使用新的优化结果作为起点"""
            pop = np.zeros((self.pop_size, self.dim))

            # 第一个粒子使用最新优化结果
            pop[0] = [
                PROB1_PARAMS["vx"], PROB1_PARAMS["vy"],
PROB1_PARAMS["t_release"], PROB1_PARAMS["t_delay"],
```

```
                INITIAL_SOLUTIONS["FY2"]["vx"],
INITIAL_SOLUTIONS["FY2"]["vy"],
                INITIAL_SOLUTIONS["FY2"]["tr"],
INITIAL_SOLUTIONS["FY2"]["td"],
                INITIAL_SOLUTIONS["FY3"]["vx"],
INITIAL_SOLUTIONS["FY3"]["vy"],
                INITIAL_SOLUTIONS["FY3"]["tr"],
INITIAL_SOLUTIONS["FY3"]["td"]
            ]
#
        # 剩余粒子：基于新初始解进行随机扰动
        for i in range(1, self.pop_size):
            # FY1 参数（基于新优化结果）
            pop[i, 0] = PROB1_PARAMS["vx"] + np.random.uniform(-5.0, 5.0)
# 缩小扰动范围，因为已有较好解
            pop[i, 1] = PROB1_PARAMS["vy"] + np.random.uniform(-5.0, 5.0)
            pop[i, 2] = PROB1_PARAMS["t_release"] + np.random.uniform(-
0.5, 0.5)
            pop[i, 3] = PROB1_PARAMS["t_delay"] + np.random.uniform(-0.5,
0.5)

            # FY2 参数（基于新优化结果）
            pop[i, 4] = INITIAL_SOLUTIONS["FY2"]["vx"] +
np.random.uniform(-3.0, 3.0)
            pop[i, 5] = INITIAL_SOLUTIONS["FY2"]["vy"] +
np.random.uniform(-3.0, 3.0)
            pop[i, 6] = INITIAL_SOLUTIONS["FY2"]["tr"] +
np.random.uniform(-0.5, 0.5)
            pop[i, 7] = INITIAL_SOLUTIONS["FY2"]["td"] +
np.random.uniform(-0.3, 0.3)

            # FY3 参数（基于新优化结果）
            pop[i, 8] = INITIAL_SOLUTIONS["FY3"]["vx"] +
np.random.uniform(-2.0, 2.0)
            pop[i, 9] = INITIAL_SOLUTIONS["FY3"]["vy"] +
np.random.uniform(-2.0, 2.0)
            pop[i, 10] = INITIAL_SOLUTIONS["FY3"]["tr"] +
np.random.uniform(-0.3, 0.3)
            pop[i, 11] = INITIAL_SOLUTIONS["FY3"]["td"] +
np.random.uniform(-0.2, 0.2)

        # 确保参数在边界内
        return np.clip(pop, self.bounds[:, 0], self.bounds[:, 1])
```

```
    def calculate_fitness(self, particle):
        """计算适应度，考虑无人机速度上限和导弹速度"""
        fy1_vx, fy1_vy, fy1_tr, fy1_td, \
            fy2_vx, fy2_vy, fy2_tr, fy2_td, \
            fy3_vx, fy3_vy, fy3_tr, fy3_td = particle

        all_intervals = []
        vz = 0.0
        uav_min_speed = UAV_SPEED_CONSTRAINTS["min"]

        # FY1 遮蔽区间
        fy1_vmag = math.sqrt(fy1_vx ** 2 + fy1_vy ** 2 + vz ** 2)
        if uav_min_speed - 1e-9 <= fy1_vmag <= self.uav_speed_max + 1e-9:
            try:
                _, (fy1_det_t, fy1_det_x, fy1_det_y, fy1_det_z) = cate_det_pos(
                    "FY1", fy1_vx, fy1_vy, fy1_tr, fy1_td
                )
                if 50.0 <= fy1_det_z <= 2000.0:
                    _, fy1_intervals = cate_se_ocion_t(
                        fy1_det_t, fy1_det_x, fy1_det_y, fy1_det_z,
self.missile_speed
                    )
                    all_intervals.extend(fy1_intervals)
            except:
                pass

        # FY2 遮蔽区间
        fy2_vmag = math.sqrt(fy2_vx ** 2 + fy2_vy ** 2 + vz ** 2)
        if uav_min_speed - 1e-9 <= fy2_vmag <= self.uav_speed_max + 1e-9:
            try:
                _, (fy2_det_t, fy2_det_x, fy2_det_y, fy2_det_z) = cate_det_pos(
                    "FY2", fy2_vx, fy2_vy, fy2_tr, fy2_td
                )
                if 50.0 <= fy2_det_z <= 2000.0:
                    _, fy2_intervals = cate_se_ocion_t(
                        fy2_det_t, fy2_det_x, fy2_det_y, fy2_det_z,
self.missile_speed
                    )
                    all_intervals.extend(fy2_intervals)
            except:
                pass

        # FY3 遮蔽区间
        fy3_vmag = math.sqrt(fy3_vx ** 2 + fy3_vy ** 2 + vz ** 2)
```

```
            if uav_min_speed - 1e-9 <= fy3_vmag <= self.uav_speed_max + 1e-9:
                try:
                    # 验证投放点/起爆点
                    (fy3_rel_x, fy3_rel_y, _), (fy3_det_t, fy3_det_x, fy3_det_y, _) =
cate_det_pos(
                        "FY3", fy3_vx, fy3_vy, fy3_tr, fy3_td
                    )
                    z_pos = 700.0 + 0.5 * (-9.8) * fy3_td ** 2
                    if 50.0 <= z_pos <= 2000.0:  # 起爆点 z 约束
                        _, fy3_intervals = cate_se_ocion_t(
                            fy3_det_t,  fy3_det_x,  fy3_det_y,  z_pos,
self.missile_speed
                        )
                        all_intervals.extend(fy3_intervals)
                except:
                    pass

            merged_time, _ = merge_inval(all_intervals)
            return merged_time if merged_time >= 1e-3 else 1e-6

        def update_velocity(self, iter_idx):
            w = self.w_start - (self.w_start - self.w_end) * (iter_idx / self.max_iter)
            r1 = np.random.rand(self.pop_size, self.dim)
            r2 = np.random.rand(self.pop_size, self.dim)

            self.vel = w * self.vel + \
                       self.c1 * r1 * (self.p_best_pos - self.pos) + \
                       self.c2 * r2 * (self.g_best_pos - self.pos)
            self.vel = np.clip(self.vel, -self.v_max, self.v_max)

        def update_position(self):
            self.pos = self.pos + self.vel
            self.pos = np.clip(self.pos, self.bounds[:, 0], self.bounds[:, 1])

        def update_best(self):
            current_fit = np.array([self.calculate_fitness(p) for p in self.pos])
            improved = current_fit > self.p_best_fit
            self.p_best_pos[improved] = self.pos[improved].copy()
            self.p_best_fit[improved] = current_fit[improved]

            current_best_idx = np.argmax(current_fit)
            if current_fit[current_best_idx] > self.g_best_fit:
                self.g_best_pos = self.pos[current_best_idx].copy()
                self.g_best_fit = current_fit[current_best_idx]
```

```
    def optimize(self, verbose=False):
        """执行优化，可控制是否打印进度"""
        if verbose:
            print(f"PSO 优化：问题 4（FY1/FY2/FY3 各投 1 枚烟幕弹）")
            print(
                f"种群规模：{self.pop_size}| 最大迭代：{self.max_iter}| 速度约束：{UAV_SPEED_CONSTRAINTS['min']}~{self.uav_speed_max}m/s")

        for i in range(self.max_iter):
            self.update_velocity(i)
            self.update_position()
            self.update_best()
            self.fit_history.append(self.g_best_fit)

            # 每 50 次迭代打印进度
            if verbose and (i + 1) % 50 == 0:
                print(f"迭代 {i + 1}/{self.max_iter} | 全局最优遮蔽时间：{self.g_best_fit:.2f}s")

        if verbose:
            print(f"优化结束！最终最优总遮蔽时间：{self.g_best_fit:.2f}s")
        return self.g_best_pos, self.fit_history
```

附录 6

problem_5.py 关键代码

```
# 无人机-导弹分配
def assign_drone_to_missile():
    assign_result = {}
    missile_list = ["M1", "M2", "M3"]
    drone_list = ["FY1", "FY2", "FY3", "FY4", "FY5"]

    for drone_id in drone_list:
        drone_pos = DRONE_INIT[drone_id]
        dist_dict = {}
        for missile_id in missile_list:
            missile_seg_p1 = MISSILE_PARAMS[missile_id]["init_pos"]
            missile_seg_p2 = (0.0, 0.0, 0.0)
            dist = point_to_segment_dist(drone_pos, missile_seg_p1, missile_seg_p2)
            dist_dict[missile_id] = dist
        target_missile = min(dist_dict.items(), key=lambda x: x[1])[0]
```

```
            assign_result[drone_id] = {
                "target_missile": target_missile,
                "distance": round(dist_dict[target_missile], 2)
            }
        return assign_result


# 3.3 烟幕投放/起爆参数计算
def calc_smoke_params(drone_id, vx, vy, t_release, t_delay):
        drone_x0, drone_y0, drone_z0 = DRONE_INIT[drone_id]
        g = -9.8   #  重力加速度

        #  投放点
        release_x = drone_x0 + vx * t_release#
        release_y = drone_y0 + vy * t_release#
        release_z = drone_z0#########
        release_pos = (round(release_x, 3), round(release_y, 3), round(release_z, 3))

        #  起爆点
        det_x = release_x + vx * t_delay#
        det_y = release_y + vy * t_delay
        det_z = release_z + 0.5 * g * (t_delay ** 2)
        t_det = t_release + t_delay
        det_pos = (round(det_x, 3), round(det_y, 3), round(det_z, 3))

        return release_pos, (t_det, det_pos)


#  决策变量约束检查
def check_constraints(vx, vy, t1, t2, t3, d1, d2, d3, drone_id):
        # 1.  速度约束
        v_mag = math.sqrt(vx ** 2 + vy ** 2)
        if not (CONSTRAINTS["drone_v_min"] - 1e-9 <= v_mag <=
CONSTRAINTS["drone_v_max"] + 1e-9):
            return False, f"速度{round(v_mag, 3)}m/s 超出 70~140m/s 范围"

        # 2.  投放间隔约束
        if t2 - t1 < CONSTRAINTS["drop_interval_min"] - 1e-9:
            return False, f"第 1-2 枚投放间隔{round(t2 - t1, 3)}s<1s"
        if t3 - t2 < CONSTRAINTS["drop_interval_min"] - 1e-9:
            return False, f"第 2-3 枚投放间隔{round(t3 - t2, 3)}s<1s"

        # 3.  起爆点 z 坐标约束
        for t_release, t_delay in [(t1, d1), (t2, d2), (t3, d3)]:
```

```
        _, (_, (_, _, det_z)) = calc_smoke_params(drone_id, vx, vy, t_release, t_delay)
        if not (CONSTRAINTS["det_z_min"] - 1e-9 <= det_z <= CONSTRAINTS["det_z_max"] + 1e-9):
            return False, f"起爆点 z={round(det_z, 3)}m 超出 50~2000m 范围"

    # 4. 时间非负约束
    if any(t < 0 for t in [t1, t2, t3]):
        return False, "投放时间不能为负"
    if any(d < 0 for d in [d1, d2, d3]):
        return False, "起爆延迟不能为负"

    return True, "约束满足"


# 模拟退火算法
class SimulatedAnnealing:#
    def __init__(self, drone_id, target_missile, max_iter=100, init_temp=100, cool_rate=0.95, seed_offset=0):
        self.drone_id = drone_id
        self.target_missile = target_missile
        self.max_iter = max_iter
        self.init_temp = init_temp
        self.cool_rate = cool_rate
        self.seed_offset = seed_offset  # 用于进程特定的随机种子
        self.decision_bounds = {
            "vx": (-140.0, 0.0),  # x 方向速度
            "vy": (-140.0, 140.0),  # y 方向速度
            "t1": (0.001, 30.0),  # 第 1 枚投放时间(s)
            "d1": (0.1, 15.0),  # 第 1 枚起爆延迟(s)
            "t2": (1.001, 35.0),  # 第 2 枚投放时间(s)
            "d2": (0.1, 15.0),  # 第 2 枚起爆延迟(s)
            "t3": (2.001, 40.0),  # 第 3 枚投放时间(s)
            "d3": (0.1, 15.0)  # 第 3 枚起爆延迟(s)
        }
        self.converge_history = []#

        # 设置进程特定的随机种子，确保可复现性
        self.local_seed = SEED + self.seed_offset
        np.random.seed(self.local_seed)

    def init_solution(self):
        while True:
            vx = np.random.uniform(*self.decision_bounds["vx"])
```

```
                vy = np.random.uniform(*self.decision_bounds["vy"])
                t1 = np.random.uniform(*self.decision_bounds["t1"])
                d1 = np.random.uniform(*self.decision_bounds["d1"])
                t2 = np.random.uniform(t1 + 1.0, self.decision_bounds["t2"][1])
                d2 = np.random.uniform(*self.decision_bounds["d2"])
                t3 = np.random.uniform(t2 + 1.0, self.decision_bounds["t3"][1])
                d3 = np.random.uniform(*self.decision_bounds["d3"])

                valid, _ = check_constraints(vx, vy, t1, t2, t3, d1, d2, d3,
self.drone_id)
                if valid:
                    return np.array([vx, vy, t1, d1, t2, d2, t3, d3])

    def generate_neighbor(self, current_sol):
        while True:
            perturb = np.array([
                np.random.uniform(-20.0, 20.0),   # vx 扰动
                np.random.uniform(-20.0, 20.0),   # vy 扰动
                np.random.uniform(-3.0, 3.0),    # t1 扰动
                np.random.uniform(-1.5, 1.5),    # d1 扰动
                np.random.uniform(-5.0, 5.0),    # t2 扰动
                np.random.uniform(-1.5, 1.5),    # d2 扰动
                np.random.uniform(-8.0, 8.0),    # t3 扰动
                np.random.uniform(-1.5, 1.5)    # d3 扰动
            ])
            new_sol = current_sol + perturb

            # 边界裁剪
            new_sol[0] = np.clip(new_sol[0], *self.decision_bounds["vx"])
            new_sol[1] = np.clip(new_sol[1], *self.decision_bounds["vy"])
            new_sol[2] = np.clip(new_sol[2], *self.decision_bounds["t1"])
            new_sol[3] = np.clip(new_sol[3], *self.decision_bounds["d1"])
            new_sol[4] = np.clip(new_sol[4], *self.decision_bounds["t2"])
            new_sol[5] = np.clip(new_sol[5], *self.decision_bounds["d2"])
            new_sol[6] = np.clip(new_sol[6], *self.decision_bounds["t3"])
            new_sol[7] = np.clip(new_sol[7], *self.decision_bounds["d3"])

            # 确保投放间隔≥1s
            new_sol[4] = max(new_sol[4], new_sol[2] + 1.0)
            new_sol[6] = max(new_sol[6], new_sol[4] + 1.0)

            # 检查约束
            vx, vy, t1, d1, t2, d2, t3, d3 = new_sol
            valid, _ = check_constraints(vx, vy, t1, t2, t3, d1, d2, d3,
```

```
self.drone_id)
                if valid:
                    return new_sol

    def calculate_fitness(self, solution):#<<<<<<<<<<<
        vx, vy, t1, d1, t2, d2, t3, d3 = solution
        smoke_details = []
        valid_count = 0  # 有效烟幕弹数量
        all_intervals = []  # 收集所有有效烟幕弹的遮挡区间（用于合并）

        for idx, (t_release, t_delay) in enumerate([(t1, d1), (t2, d2), (t3, d3)], 1):
            release_pos, (t_det, (det_x, det_y, det_z)) = calc_smoke_params(
                self.drone_id, vx, vy, t_release, t_delay
            )
            shield_intervals, shield_time = cal_ful_ocion_t(
                t_det, det_x, det_y, det_z, self.target_missile
            )

            # 判断是否为有效烟幕弹（有遮挡时间）
            is_valid = shield_time > REWARD_CONFIG["valid_threshold"]
            if is_valid:
                valid_count += 1
                all_intervals.extend(shield_intervals)  # 仅收集有效弹的区间

            # 记录单枚烟幕弹详情
            smoke_details.append({
                "bomb_idx": idx,
                "release_time": round(t_release, 3),
                "release_pos": release_pos,
                "det_delay": round(t_delay, 3),
                "det_time": round(t_det, 3),
                "det_pos": (det_x, det_y, det_z),
                "shield_intervals": [(round(s, 2), round(e, 2)) for s, e in shield_intervals],  # 单弹原始区间
                "shield_time": shield_time,  # 单弹原始遮挡时间
                "is_valid": is_valid
            })

        # 合并所有有效弹的遮挡区间，计算真实总遮挡时间（去重）
        merged_intervals = merge_intervals(all_intervals)
        total_shield = sum(end - start for start, end in merged_intervals)  # 合并后的总遮挡时间
```

```
        # 计算奖励
        base_reward = valid_count * REWARD_CONFIG["per_smoke_reward"]
        bonus_reward = max(0, valid_count - 1) * REWARD_CONFIG["multi_bonus"]
        total_reward = base_reward + bonus_reward

        # 最终适应度 = 合并后的总遮挡时间 + 奖励
        total_fitness = total_shield + total_reward

        return total_fitness, smoke_details, valid_count, total_reward, merged_intervals

    def optimize(self):
        print(f"开始优化无人机{self.drone_id}（干扰{self.target_missile}）")
        print(f"SA 参数：初始温度={self.init_temp}，降温系数={self.cool_rate}，总迭代={self.max_iter}")
        print(f"奖励机制：每枚有效弹{REWARD_CONFIG['per_smoke_reward']}s，多弹叠加{REWARD_CONFIG['multi_bonus']}s/枚")

        current_sol = self.init_solution()
        current_fitness, _, _, _, _ = self.calculate_fitness(current_sol)
        best_sol = current_sol.copy()
        best_fitness = current_fitness
        best_smoke_details = None
        best_valid_count = 0
        best_total_reward = 0
        best_merged_intervals = []  # 记录最优解的合并区间
        temp = self.init_temp

        for iter_idx in range(self.max_iter):
            for _ in range(50):
                new_sol = self.generate_neighbor(current_sol)
                new_fitness, new_smoke_details, new_valid_count, new_total_reward, new_merged_intervals = self.calculate_fitness(
                    new_sol)

                delta_f = new_fitness - current_fitness
                if delta_f > 1e-9:
                    current_sol = new_sol.copy()
                    current_fitness = new_fitness
                    if current_fitness > best_fitness:
                        best_fitness = current_fitness
                        best_sol = current_sol.copy()
```

```
                                best_smoke_details = new_smoke_details
                                best_valid_count = new_valid_count
                                best_total_reward = new_total_reward
                                best_merged_intervals = new_merged_intervals    # 保存最优合并区间
                        else:
                            prob = math.exp(delta_f / temp)
                            if np.random.random() < prob:
                                current_sol = new_sol.copy()
                                current_fitness = new_fitness

                    self.converge_history.append(best_fitness)
                    if (iter_idx + 1) % 50 == 0:
                        # 打印时使用合并后的遮挡时间（更准确）
                        base_shield = best_fitness - best_total_reward
                        print(
                            f"SA 迭代{iter_idx + 1:3d}/{self.max_iter} | 最优适应度：{best_fitness:.3f} | "
                            f"遮挡{base_shield:.3f}s | 有效弹{best_valid_count}枚 | 当前温度：{temp:.2f}")

                    temp *= self.cool_rate

                # 重新计算最优解的详细信息（确保一致性）
                _, best_smoke_details, best_valid_count, best_total_reward, best_merged_intervals = self.calculate_fitness(
                    best_sol)
                vx, vy = best_sol[0], best_sol[1]
                v_mag = math.sqrt(vx ** 2 + vy ** 2)
                base_shield_time = best_fitness - best_total_reward  # 合并后的遮挡时间

                print(f"\n 无人机{self.drone_id}最优策略详细信息")
                print(f"最优速度：vx={round(vx, 3)}m/s, vy={round(vy, 3)}m/s, 合速度={round(v_mag, 3)}m/s, 方向角{180+np.arctan(vy/(vx+1e-9))*180/np.pi:.2f}°")
                print(f"遮挡{base_shield_time:.3f}s")
                # 打印合并后的总区间
                print(
                    f"所有有效弹合并后遮挡区间：{[(round(s, 2), round(e, 2)) for s, e in best_merged_intervals] if best_merged_intervals else '无'}")

                for smoke in best_smoke_details:
                    status = "有效" if smoke["is_valid"] else "无效"
                    print(f"\n 第{smoke['bomb_idx']}枚烟幕弹（{status}）")
```

```
                print(f"投放时间: {smoke['release_time']}s")
                print(f"投放点坐标: {smoke['release_pos']}")
                print(f"起爆延迟: {smoke['det_delay']}s")
                print(f"起爆时间: {smoke['det_time']}s")
                print(f"起爆点坐标: {smoke['det_pos']}")
                print(f"单弹遮挡区间: {smoke['shield_intervals'] if smoke['shield_intervals'] else '无'}")
                print(f"单弹遮挡时间: {smoke['shield_time']:.3f}s")

        return {
                "drone_id": self.drone_id,
                "target_missile": self.target_missile,
                "best_velocity": {"vx": round(vx, 3), "vy": round(vy, 3), "v_mag": round(v_mag, 3)},
                "best_fitness": round(best_fitness, 3),
                "base_shield_time": round(base_shield_time, 3),  # 合并后的遮挡时间
                "total_reward": round(best_total_reward, 3),
                "valid_smoke_count": best_valid_count,
                "smoke_details": best_smoke_details,
                "merged_intervals": best_merged_intervals,  # 合并后的区间
                "converge_history": self.converge_history
        }#######>>>>>>>>>>>>>>>>>


# 多进程优化函数
def optimize_drone(drone_id, target_missile, seed_offset):
    sa = SimulatedAnnealing(
        drone_id=drone_id,
        target_missile=target_missile,
        max_iter=1000,
        init_temp=10000,
        cool_rate=0.98,
        seed_offset=seed_offset
    )
    return sa.optimize()
```